

PDR



PRD · Plataforma de operaciones Exodus (EOP)

ID del documento: PRD-EOP-2077-001

Estado: Aprobado — En desarrollo **Producto:**

Plataforma Exodus Ops (EOP) **Cliente:** División de
Reconstrucción de Vault-Tec

Ingeniería: Exodus Systems Inc. — División de Ingeniería **Ciclo:** 1.º-2.º

trimestre · 16 semanas · 4 equipos (3 ingenieros cada uno) **Clasificación:**

Interno — Solo ingeniería

Índice

1. [Descripción general del producto](#)
 2. [Perfiles de usuario](#)
 3. [v0.1 — EOP v0.1 · Capa de comunicación central](#)
 4. [v0.2 — EOP v0.2 · Motor de procesamiento HPC](#)
 5. [v0.3 — EOP v0.3 · API pública y entrega segura](#)
 6. [v1.0 — EOP v1.0 · Integración periférica y entrega de productos](#)
 7. [Requisitos no funcionales](#)
 8. [Negociación de prototipos — Contrato entre equipos](#)
 9. [Contrato de ingeniería](#)
-

1. Descripción general del producto

1.1 Planteamiento del problema

La División de Reconstrucción de Vault-Tec opera una infraestructura crítica distribuida compuesta por múltiples nodos de búnker conectados a través de una red privada. La situación actual presenta cinco problemas operativos sin resolver:

#	Problema	Impacto operativo	Línea de base actual
1	No hay comunicación confiable entre nodos	Los búnkeres no pueden coordinarse. Se pierden comandos. El estado de la red es inconsistente.	MTTR > 4 horas
2	Procesamiento de datos secuencial de un solo hilo	El análisis de la topología de la red lleva horas. Las decisiones operativas llegan demasiado tarde.	< 1K nodos/min
3	No existe una API pública para la integración externa.	La IA y los servicios corporativos no pueden consultar ni controlar la red.	0 integraciones activas
4	Los dispositivos periféricos se desconectaron del sistema.	Los sensores ubicados en búnkeres remotos no envían datos de telemetría al sistema central	Cobertura de bordes del 0 %
5	No hay observabilidad del sistema	El diagnóstico de fallos es manual. No hay métricas ni alertas automatizadas.	MTTI > 60 min

1.2 Visión del producto

EOP es una plataforma de sistemas distribuidos nativa de la nube que permite a Vault-Tec coordinar, procesar y observar su infraestructura distribuida en tiempo real, con alta disponibilidad, capacidades de computación paralela e integración con servicios externos y dispositivos periféricos.

1.3 Metas y no metas

En el alcance: EOP v1.0 ofrece:

- Servidor TCP en C++ con comunicación entre procesos a nivel de socket, compatible con N clientes concurrentes (N > 10k).
- Biblioteca de cliente en C que encapsula el protocolo de comunicación
- Motor de procesamiento HPC paralelo (OpenMP) para el análisis de grafos de red.
- Implemente una puerta de enlace API REST con patrones de resiliencia (circuit breaker, reintentos, tiempos de espera).
- Ingesta de telemetría perimetral mediante MQTT + ZephyrOS en ESP32
- Pila completa de observabilidad: trazas distribuidas de OTel, panel de control de SLO/SLA, registros centralizados.
- Despliegue de Kubernetes con lanzamiento canario y entrega azul-verde

Fuera de alcance: no está integrado en la versión 1.0:

- Interfaz gráfica de usuario (GUI / panel visual)
- Autenticación mutua mTLS entre servicios internos
- ML/IA para la detección predictiva de fallas
- Orquestación multirregional o multinube
- Algoritmos de consenso distribuido en código (Raft, Paxos)

- Compatibilidad con hardware heredado (anterior a 2015)
- Gestión multitenant con roles granulares

1.4 Métricas de éxito

Métricas	Definición	Punto de partida	Objetivo (EOP v1.0)
MTTR (North Star)	Tiempo transcurrido desde el fallo del nodo hasta la recuperación automática	> 4 horas	< 200 ms
Tasa de intervención manual	% de incidentes resueltos sin intervención humana	~0 % automatizado	> 80 %
Disponibilidad del servicio	Medido por la sonda Prometheus cada 30 segundos.	No medido	≥ 99,9 % mensual
Presupuesto de error mensual	Presupuesto de error restante en el momento del lanzamiento	No medido	> 0 en el momento de la entrega
Latencia P99 de la API REST	Latencia de extremo a extremo hasta el cliente externo	No medido	< 300 ms
Latencia de extremo a extremo en el borde	ESP32 → servidor EOP, medida a través de OTel	No medido	< 500 ms P99

2. Perfiles de usuario

Perfil	Rol y contexto	Objetivo principal	Problema actual
Operador de bóveda	Ingeniero de operaciones en Vault-Tec. Trabaja desde un terminal Linux. Gestiona varios búnkeres simultáneamente.	Detectar y resolver fallas en los nodos sin intervención manual.	El diagnóstico manual lleva horas. No hay visibilidad centralizada del estado de la red.
Analista de redes	Analista de infraestructura. Necesita procesar topologías de red a gran escala para detectar cuellos de botella y nodos críticos.	Ejecuta análisis de grafos en la red del búnker y recibe los resultados en un tiempo razonable.	El procesamiento secuencial tarda horas en grafos de más de 50 000 nodos.

Persona	Función y contexto	Objetivo principal	Problema actual
Ingeniero de integración	Ingeniero de integración en Vault-Tec. Conecta servicios externos (IA, sistemas corporativos) a EOP a través de API.	Consume el estado de la red y envía comandos desde sistemas externos de manera confiable y observable.	No existe una API pública. Las integraciones se realizan manualmente con acceso directo a los nodos.
Dispositivo periférico	ESP32 con ZephyrOS en búnkeres remotos. Conectividad intermitente.	Envía datos de telemetría al sistema central de manera confiable, incluso con mala conectividad.	Los sensores se desconectan del sistema. Se pierden datos.

3. Hito 1 — EOP v0.1 · Capa de comunicación principal

Objetivo de la versión: Un operador de bóveda puede registrar nodos de búnker, enviar comandos a través de la red y recibir respuestas con una observabilidad básica del sistema, todo ello sin intervención manual en la capa de comunicación.

Alcance del equipo: 3 ingenieros, ~4 semanas. Cada equipo tiene total flexibilidad para definir la granularidad del servicio. El contrato del protocolo se negocia colectivamente en la Reunión de Negociación del Protocolo (Semana 2).

Decisión arquitectónica: Sockets como mecanismo IPC: El servidor C++ utiliza sockets BSD/POSIX directamente (socket, bind, listen, accept, recv, send). Ningún marco RPC abstrae el descriptor de archivo. Los clientes C utilizan la misma API de sockets. Esto es intencional: el objetivo principal de la versión 0.1 es que los ingenieros dominen el modelo de comunicación en la capa de transporte: gestión de descriptors de archivo, concurrencia sobre sockets, encapsulación de mensajes y manejo de errores de red. El protocolo de mensajes (tipos, encapsulación, serialización) es diseñado por cada equipo y documentado en un ADR. El acuerdo entre equipos se negocia en la Reunión de Negociación de Prototipos.

Epic E1 — Servidor TCP en C++

US-101 · Conexión de nodo y saludo de bienvenida

Como operador de bóveda, quiero conectarme al servidor EOP desde cualquier nodo de la red utilizando el cliente C, para poder enviar comandos y recibir respuestas sin necesidad de conocer los detalles de la implementación del servidor.

Criterios de aceptación:

- **AC1.** El servidor escucha en un puerto TCP configurable mediante la variable de entorno **EOP_PORT** (valor predeterminado: 9026). Si el puerto ya está en uso, el servidor no se inicia y muestra un mensaje de error descriptivo.

- **AC2.** Un cliente C puede establecer una conexión usando ``connect()`` al `host:puerto` y recibir un mensaje de bienvenida dentro de los 500 ms posteriores al establecimiento de la conexión.
- **AC3.** El mensaje de bienvenida incluye: versión del servidor, marca de tiempo del servidor y un ID de sesión asignado para la conexión.
- **AC4.** La conexión permanece abierta hasta que el cliente la cierra explícitamente o hasta que expira el tiempo de espera de inactividad configurable (por defecto: 30 s, configurable mediante ``EOP_IDLE_TIMEOUT``).
- **AC5.** Si el servidor no está disponible, el cliente recibe un error claro: sin bloqueo, sin cuelgue indefinido, sin fallo silencioso.
- **AC6.** El host y el puerto son los únicos parámetros necesarios para conectarse. No se necesita ninguna configuración adicional en el lado del cliente.
- **AC7.** El servidor registra cada nueva conexión con: marca de tiempo, IP del cliente, ID de sesión asignado y número total de conexiones activas.
- **AC8.** Una prueba de integración verifica que el ciclo completo de conexión → establecimiento de conexión → desconexión se complete sin errores bajo ejecución repetida (100 iteraciones).

US-102 · Soporte simultáneo para múltiples clientes

Como operador de bóveda, quiero tener varios clientes conectados simultáneamente sin que ninguno experimente tiempos de respuesta degradados, para poder operar varios búnkeres en paralelo desde diferentes terminales.

Criterios de aceptación:

- **AC1.** El servidor admite un mínimo de 100 clientes concurrentes sin tiempos de espera ni errores en ninguno de ellos.
- **AC2.** La latencia de respuesta para cualquier cliente activo no aumenta más del doble cuando hay 100 clientes conectados en comparación con 1 cliente (medida bajo la misma carga de trabajo).
- **AC3.** Un cliente que se desconecta abruptamente (proceso terminado con SIGKILL) no afecta a ningún otro cliente conectado: no hay fallos, ni respuestas bloqueadas, ni fugas de recursos.
- **AC4.** El servidor no tiene fugas de descriptores de archivo: verificado comparando el recuento de `/proc/PID/fd` antes y después de 1 millón de ciclos de conexión/desconexión.
- **AC5.** El modelo de concurrencia (hilo por conexión, grupo de hilos o selección asíncrona/epoll) está documentado en un ADR con un análisis de compensación justificado.
- **AC6.** ThreadSanitizer no informa de condiciones de carrera de datos al ejecutar el conjunto de pruebas de clientes concurrentes.
- **AC7.** El servidor impone un límite máximo de conexiones configurable (`EOP_MAX_CLIENTS`). Las conexiones que superen este límite se rechazan con un mensaje de error descriptivo, en lugar de descartarse silenciosamente.
- **AC8.** Una prueba de carga con 1000 clientes que envían 10 000 solicitudes cada uno se completa sin errores del servidor ni tiempos de espera agotados por parte de los clientes.

US-103 · Registro de nodos

Como operador de bóveda, quiero registrar un nodo de búnker con sus metadatos y recibir una confirmación, para que el sistema mantenga un registro centralizado de qué nodos están activos en la red.

Criterios de aceptación:

- **AC1.** El cliente envía un mensaje REGISTER con los campos: node_id (cadena única), bunker_name, ip_address y capacity.
- **AC2.** El servidor responde con ACK, incluyendo el node_id asignado, en un plazo de 200 ms tras recibir la solicitud.
- **AC3.** Si el node_id ya existe, el servidor responde con ERROR_DUPLICATE y una descripción legible; no se bloquea ni se sobrescribe la entrada existente.
- **AC4.** El registro se mantiene en memoria (la persistencia no es necesaria en la versión 0.1). El registro se conserva incluso después de que se cierre la conexión del cliente.
- **AC5.** Se puede consultar un nodo registrado mediante su node_id utilizando un mensaje QUERY_NODE. La respuesta incluye todos los metadatos registrados.
- **AC6.** Si se consulta un node_id inexistente, se devuelve ERROR_NOT_FOUND, no un bloqueo del servidor ni una respuesta vacía.
- **AC7.** El mensaje LIST_NODES devuelve todos los nodos registrados actualmente, junto con su estado y metadatos.
- **AC8.** Las pruebas unitarias abarcan: registro exitoso, registro duplicado, consulta de nodos existentes, consulta de nodos inexistentes y lista de múltiples nodos.

US-104 · Detección de desconexión de nodos y actualización de estado

Como operador de Vault, quiero recibir notificaciones cuando un nodo se desconecte y que su estado se actualice automáticamente, para no tener que enviar comandos a nodos desconectados y poder ver siempre un estado de red preciso.

Criterios de aceptación:

- **AC1.** El servidor detecta la desconexión del cliente en 10 segundos mediante latidos periódicos o mantenimiento de conexión TCP (mecanismo documentado en ADR).
- **AC2.** Al detectar la desconexión, el nodo pasa al estado OFFLINE en el registro interno.
- **AC3.** La consulta QUERY_NODE en un nodo OFFLINE devuelve el estado correcto: ni un error de búsqueda ni datos ONLINE obsoletos.
- **AC4.** El servidor registra el evento de desconexión con: marca de tiempo, ID del nodo, ID de sesión y duración de la conexión
- **AC5.** Un nodo puede reconectarse y registrarse nuevamente sin necesidad de reiniciar el servidor. Su estado vuelve a ONLINE tras un registro exitoso.

- **AC6.** El intervalo de latidos es configurable mediante `EOP_HEARTBEAT_INTERVAL` (predeterminado: 5 s). El servidor tolera un latido perdido antes de marcar el nodo como OFFLINE.
- **AC7.** Una prueba de integración verifica el ciclo completo: registro → establecimiento de latido → finalización del proceso del cliente → verificación del estado OFFLINE → reconexión → verificación del estado ONLINE.

Epic E2 — Biblioteca de clientes C

US-105 · Abstracción de conexión

Como ingeniero de integración, quiero utilizar una función `eop_connect(host, port)` que abstraiga la conexión TCP, para poder integrar EOP en mis herramientas sin tener que escribir código de gestión de sockets directamente.

Criterios de aceptación:

- **AC1.** ``eop_connect(host, port)`` devuelve un identificador opaco ``eop_client_t*`` si la operación se realiza correctamente.
- **AC2.** Si falla, devuelve ``NULL`` y establece un código de error recuperable, accesible mediante ``eop_last_error()``, con una descripción legible.
- **AC3.** La función tiene un tiempo de espera de conexión configurable. Nunca se bloquea indefinidamente.
- **AC4.** ``eop_client.h`` expone únicamente la API pública. Los detalles internos (descriptor de archivo, búferes, estado interno) están completamente ocultos tras el identificador opaco.
- **AC5.** La biblioteca se compila como biblioteca compartida (.so) y como biblioteca estática (.a) a partir del mismo código fuente, sin necesidad de modificar el código
- **AC6.** El archivo de cabecera público es compatible con C99 y se puede usar tanto en C como en C++ sin modificaciones.
- **AC7.** Un ejemplo de uso en ``examples/connect_basic.c`` se compila y ejecuta correctamente en un servidor en producción.
- **AC8.** Una prueba unitaria verifica que la conexión `eop_connect` a un host inaccesible falle correctamente dentro del tiempo de espera establecido, sin fugas de recursos (verificado con AddressSanitizer).

US-106 · Envío de comandos y recepción de respuestas

Como ingeniero de integración, quiero enviar un comando y recibir la respuesta con una sola llamada a una función, para poder integrar EOP en los scripts sin tener que gestionar manualmente el formato de los mensajes.

Criterios de aceptación:

- **AC1.** `eop_send_command(client, cmd_type, payload, len)` envía un mensaje y bloquea el hilo de llamada hasta que se recibe una respuesta.
- **AC2.** Devuelve un `eop_response_t` con: `status_code, payload, payload_len`. El autor de la llamada es responsable de liberarlo mediante `eop_response_free(response)`.
- **AC3.** Si el servidor cierra la conexión durante la espera, la función devuelve `EOP_ERR_DISCONNECTED`: sin bloqueo, sin comportamiento indefinido.

- **AC4.** Si el servidor no responde dentro del tiempo de espera de comando configurado, la función devuelve `EOP_ERR_TIMEOUT`.
 - **AC5.** La función es segura para subprocesos: varios subprocesos pueden invocarla simultáneamente en el mismo identificador con un comportamiento bien definido.
 - **AC6.** Llamar a `eop_response_free(NULL)` es una operación segura sin efecto — no se produce ningún fallo.
 - **AC7.** Valgrind `--leak-check=full` reporta cero fugas en una prueba que ejecuta 1,000 secuenciales Ciclos «`send_command`» → «`response_free`».
 - **AC8.** Las pruebas unitarias cubren: comando/respuesta exitosos, desconexión del servidor durante la espera, vencimiento del tiempo de espera y manejo de errores de manejador nulo.
-

US-107 · Desconexión limpia y liberación de recursos

Como ingeniero de integración,
quiero desconectarme del servidor y que todos los recursos se liberen automáticamente, **para que** los clientes de larga duración no acumulen descriptores de archivos ni fugas de memoria.

Criterios de aceptación:

- **AC1.** `eop_disconnect(client)` llama a `shutdown(SHUT_RDWR)` antes de `close()` en el socket, asegurando que el servidor reciba el segmento `FIN`.
 - **AC2.** `eop_disconnect` libera toda la memoria asociada con el identificador: búferes, estado interno y el propio identificador.
 - **AC3.** Llamar a ``eop_disconnect(NULL)`` es una operación segura que no produce ningún efecto: no provoca fallos ni comportamientos indefinidos.
 - **AC4.** Después de `eop_disconnect`, el uso del identificador se documenta como comportamiento indefinido en el encabezado público.
 - **AC5.** AddressSanitizer no reporta ningún error en una prueba que ejecuta 1,000 ciclos de `connect` → `send` → `disconnect` de forma secuencial.
 - **AC6.** El servidor detecta correctamente la desconexión limpia (a través de `recv` que devuelve 0) y marca el nodo como `OFFLINE` dentro de la ventana de latido.
 - **AC7.** Una prueba concurrente con 10 subprocesos, cada uno de los cuales ejecuta 100 ciclos de conexión → envío → desconexión, se completa sin errores con ThreadSanitizer.
-

Epic E3 — Contenedorización y Kubernetes

US-108 · Implementación reproducible

Como operador de Vault,
quiero implementar la pila completa de EOP con un solo comando en cualquier máquina,
para que Vault-Tec pueda reproducir el entorno sin necesidad de instalar dependencias manualmente.

Criterios de aceptación:

- **AC1.** El servidor cuenta con un archivo `Dockerfile` de varias etapas: una etapa de `compilación` con el compilador de C++ y una etapa de `tiempo de ejecución` sin él. La imagen de tiempo de ejecución es ≤ 50 MB.

- **AC2.** `ENTRYPOINT` utiliza el formato de matriz JSON. El proceso C++ tiene el PID 1 o se ejecuta bajo un proceso `init` adecuado que propaga `SIGTERM`.
 - **AC3.** `SIGTERM` desencadena un apagado ordenado: el servidor deja de aceptar nuevas conexiones, espera a que finalicen las conexiones activas (máximo 15 s) y sale con el código 0.
 - **AC4.** El comando `«docker compose up»` inicia la pila completa (servidor + OTel Collector + SigNoz) y todos los servicios superan sus comprobaciones de estado en menos de 60 segundos.
 - **AC5.** Se incluyen los manifiestos de Kubernetes (`Deployment` + `Service`). `kubectl apply -f k8s/` implementa el servidor sin configuración adicional.
 - **AC6.** Toda la configuración se inyecta a través de variables de entorno. No existen valores codificados en la imagen ni en los propios manifiestos de K8s (utiliza `ConfigMap`).
 - **AC7.** La imagen se compila en CI con cada solicitud de extracción. Una compilación fallida bloquea la fusión.
 - **AC8.** Un objetivo `«deploy-local»` en el `Makefile` ejecuta `«docker compose up»` y realiza una prueba básica para verificar que el servidor acepta conexiones.
-

US-109 · Recuperación automática de pods

Como operador de Vault,

quiero que el pod del servidor se reinicie automáticamente si el proceso falla,

para que la disponibilidad del servicio no dependa de que alguien esté vigilando el terminal.

Criterios de aceptación:

- **AC1.** El pod tiene `restartPolicy: Always` y una prueba de actividad (comprobación TCP al puerto del servidor cada 10 s, `failureThreshold: 3`).
 - **AC2.** El Pod cuenta con una prueba de disponibilidad (comprobación TCP del puerto del servidor). Se marca como `NotReady` hasta que el servidor esté listo para aceptar conexiones.
 - **AC3.** Si el proceso del servidor falla, Kubernetes lo reinicia. El tiempo desde la falla hasta que el Pod *está en ejecución* (medido) es < 10 segundos.
 - **AC4.** Se puede acceder a los registros del Pod a través de `kubectl logs` e incluyen el motivo del fallo, si se produjo alguno.
 - **AC5.** Un cliente que tenga una conexión activa cuando el servidor se bloquee recibirá un error de desconexión claro, sin que se produzca un bloqueo silencioso.
 - **AC6.** Después de que el pod se reinicie, un cliente puede volver a conectarse y registrarse de nuevo sin ninguna intervención manual.
 - **AC7.** Una prueba de integración verifica el ciclo: `kubectl delete pod` → el Pod se reinicia → el cliente se vuelve a conectar → registra el nodo correctamente.
-

Epic E4 — Observabilidad básica

US-110 · Rastros distribuidos en SigNoz

Como operador de Vault,

Quiero ver el flujo de operaciones en el servidor como trazas distribuidas en SigNoz, **para** poder diagnosticar la latencia o los errores sin tener que revisar los registros línea por línea.

Criterios de aceptación:

- **AC1.** El servidor emite un span de OTel por cada operación relevante: `REGISTER`, `QUERY_NODE`, `LIST_NODES`, conexión de cliente, desconexión de cliente.
 - **AC2.** Cada span incluye: `operation.name`, `node.id` (si corresponde), `duration_ms`, `result` (`OK` o `ERROR`) y `session.id`.
 - **AC3.** Los spans son visibles en SigNoz en un plazo de 30 segundos desde que se produce la operación.
 - **AC4.** Los registros del servidor se envían a la salida estándar (stdout) en formato JSON estructurado con los siguientes campos: `timestamp`, `level`, `trace_id`, `span_id`, `node_id`, `event`, `message`.
 - **AC5.** El `trace_id` del registro coincide con el `trace_id` del span correspondiente en SigNoz, lo que permite la correlación entre registros y trazas a partir de la versión v0.1.
 - **AC6.** Una prueba de integración verifica que una operación `REGISTER` genere exactamente un span en SigNoz con el `operation.name` y el `result` correctos.
 - **AC7.** El OTel Collector está configurado como un contenedor independiente en `docker-compose.yml`. Exporta a SigNoz. No se pierde telemetría si SigNoz no está disponible temporalmente (búfer configurado).
 - **AC8.** Una sección del manual de procedimientos del archivo README describe cómo pasar de un `node_id` específico en SigNoz a las líneas de registro que corresponden a sus operaciones.
-

4. Hito 2 — EOP v0.2 · Motor de procesamiento HPC

Objetivo del lanzamiento: Un analista de redes puede solicitar un análisis paralelo de la topología de la red de búnkeres y recibir los resultados en segundos. Los SLO del servicio se definen, miden y visualizan en Grafana.

Hilo de continuidad:

El motor HPC no es un proceso independiente. Recibe datos de topología de red del servidor C++ v0.1 a través del protocolo de socket negociado. El flujo de extremo a extremo debe funcionar: Cliente C → v0.1 → motor HPC → envío del resultado al cliente. Si este flujo no funciona, la versión v0.2 no está completa.

Epic E5 — Motor HPC paralelo (C++/OpenMP)

US-201 · Solicitud de análisis de grafos en paralelo

Como analista de redes,

quiero enviar un gráfico de topología de red a EOP y recibir un análisis de nodos críticos,

para poder identificar cuellos de botella en la red de búnkeres de Vault-Tec sin procesar los datos manualmente.

Criterios de aceptación:

- **AC1.** El sistema acepta gráficos de topología de red en el formato definido en el protocolo v0.1 (lista de adyacencias o matriz, según se especifica en el ADR del equipo).
- **AC2.** El motor calcula: Dijkstra paralelo (ruta más corta), componentes conectados y análisis de centralidad.

- **AC3.** Para gráficos de hasta 10 000 nodos, los resultados se devuelven al cliente solicitante en menos de 5 segundos con 4 subprocesos.
 - **AC4.** La respuesta incluye: resultados del algoritmo, tamaño del gráfico (nodos, aristas), tiempo de procesamiento (ms) y número de subprocesos utilizados.
 - **AC5.** El motor registra por solicitud: tamaño del grafo recibido, algoritmos ejecutados, tiempo de procesamiento real y número de subprocesos.
 - **AC6.** Las cargas útiles de grafos no válidas o con formato incorrecto devuelven una respuesta de error estructurada, en lugar de provocar un fallo del servidor.
 - **AC7.** Las pruebas unitarias cubren: gráfico pequeño (10 nodos), gráfico mediano (1000 nodos), gráfico desconectado (múltiples componentes) y gráfico con un solo nodo.
 - **AC8.** Una prueba de integración verifica el flujo completo de extremo a extremo: cliente C → servidor v0.1 → motor HPC → resultado entregado al cliente.
-

US-202 · Paralelismo y aceleración de OpenMP

Como analista de redes,

Quiero que el motor procese grandes grafos en paralelo utilizando varios núcleos de CPU, **de modo que** el análisis de toda la red de Vault-Tec tarde segundos, en lugar de horas.

Criterios de aceptación:

- **AC1.** El motor utiliza OpenMP para la paralelización. El número de subprocesos se puede configurar mediante `OMP_NUM_THREADS` sin necesidad de recompilar.
 - **AC2.** Con 4 subprocesos, la aceleración es $\geq 2\times$ en comparación con 1 subproceso en grafos de $\geq 50\,000$ nodos (medido y reportado en el Informe de Benchmark).
 - **AC3.** Con 8 subprocesos, la aceleración es $\geq 3\times$ en comparación con 1 subproceso en grafos de $\geq 100\,000$ nodos.
 - **AC4.** El motor no presenta ninguna carrera de datos: ThreadSanitizer no reporta errores con ninguna configuración de subprocesos, desde 1 hasta `$(nproc)`.
 - **AC5.** El informe de referencia es un entregable: documenta las mediciones de aceleración en 1, 2, 4, 8 hilos en tres tamaños de grafos (10 000, 50 000 y 100 000 nodos), e incluye un análisis de la Ley de Amdahl que identifica la fracción en serie.
 - **AC6.** El informe de referencia incluye una conclusión sobre «cuándo no paralelizar»: el tamaño mínimo del grafo en el que la paralelización produce beneficios.
 - **AC7.** Las pruebas de benchmark son automatizadas y reproducibles: la orden `«make benchmark»` genera el informe sin pasos manuales.
 - **AC8.** El motor no consume más CPU de lo que especifica `OMP_NUM_THREADS`. Verificado mediante la observación del uso de la CPU con `top` o `htop` durante la ejecución de una prueba de rendimiento.
-

US-203 · Integración con el servidor v0.1

Como operador de Vault,

quiero activar el análisis de red a través de la misma interfaz que utilizo para registrar nodos, **de modo que** la capacidad de HPC forme parte del flujo de trabajo de EOP, y no sea una herramienta independiente.

Criterios de aceptación:

- **AC1.** El motor HPC se registra en el servidor v0.1 al iniciarse, utilizando el mismo `mensaje REGISTER` que cualquier nodo EOP.
 - **AC2.** Un cliente C envía un mensaje `ANALYZE_GRAPH` al servidor v0.1, el cual lo remite al motor HPC; no se requiere una conexión directa entre el cliente y el motor.
 - **AC3.** El servidor v0.1 enruta la solicitud de análisis al motor y devuelve el resultado al cliente de origen.
 - **AC4.** Si el motor HPC está desconectado, el servidor v0.1 devuelve un `ERROR_SERVICE_UNAVAILABLE` — sin fallos ni bloqueos.
 - **AC5.** Los tramos de OTel cubren toda la ruta de envío: tramo de solicitud del cliente → tramo de envío del servidor → tramo de procesamiento del motor. Los tres son visibles y están vinculados en SigNoz.
 - **AC6.** Una prueba de integración verifica lo siguiente: el motor está fuera de línea → el cliente recibe un error → el motor vuelve a estar en línea → el cliente vuelve a intentarlo → se entrega el resultado.
 - **AC7.** La extensión del protocolo para `ANALYZE_GRAPH` se documenta como un apéndice al ADR del protocolo v0.1, con la justificación del equipo.
 - **AC8.** La integración no introduce cambios que rompan la compatibilidad con ningún mensaje v0.1 existente. `buf breaking` (o la herramienta de comparación de protocolos) confirma la compatibilidad con versiones anteriores.
-

Épica E6 — Gestión de cargas de trabajo de Kubernetes

US-204 · Pila completa en Kubernetes

Como operador de Vault,

Quiero ejecutar la pila completa de EOP v0.2 en Kubernetes con todos los servicios gestionados por el clúster, **de modo que** no tenga que gestionar los procesos manualmente.

Criterios de aceptación:

- **AC1.** Todos los servicios de EOP (servidor v0.1 + motor HPC) tienen manifiestos `de implementación` de K8s con sondas de `actividad y disponibilidad`.
 - **AC2.** Todas las `implementaciones` tienen `defined resources.requests` y `resources.limits` para CPU y memoria.
 - **AC3.** `kubectl get pods --all-namespaces` muestra todos los pods de EOP en estado `Running` después de ejecutar `kubectl apply`.
 - **AC4.** Un `ConfigMap` gestiona toda la configuración del servicio (puertos, tiempos de espera, número de subprocesos). No hay valores fijos en las imágenes de contenedor.
 - **AC5.** El motor HPC puede ejecutarse como un `trabajo` de K8s (para análisis por lotes de una sola vez) o como una `implementación` (para disponibilidad continua). Ambos modos están documentados.
 - **AC6.** Un objetivo `Makefile «make k8s-apply»` aplica todos los manifiestos y espera a que todos los pods alcancen el estado `Ready`, con un tiempo de espera de 3 minutos.
 - **AC7.** El comando `kubectl rollout restart deployment/eop-server` se completa sin que se observen errores 5xx en los clientes activos (verificado con una prueba de carga simultánea).
 - **AC8.** Todos los manifiestos de K8s superan la validación `de kubeval` o `kubeconform` en la integración continua (CI). Los manifiestos no válidos bloquean la fusión.
-

Epic E7 — Panel de control de SLO y observabilidad

US-205 · Panel de control de SLO en Grafana

Como operador de Vault,

quiero ver los SLO del sistema en tiempo real en Grafana,

para saber en todo momento si EOP está cumpliendo su contrato de disponibilidad con Vault-Tec.

Criterios de aceptación:

- **AC1.** Grafana cuenta con un panel de control que muestra: el porcentaje de tiempo de actividad, el presupuesto de errores restante, la latencia P99 por servicio y las solicitudes por segundo.
 - **AC2.** El SLO de disponibilidad $\geq 99,9$ % se muestra como un indicador con umbrales claros en verde, amarillo y rojo.
 - **AC3.** Se activa una alerta cuando el presupuesto de errores cae por debajo del 50 %. La alerta es visible en Grafana y en los registros del servidor.
 - **AC4.** Prometheus recopila métricas de todos los pods a través de anotaciones de K8s cada 15 segundos.
 - **AC5.** El panel de control abarca, de forma predeterminada, un intervalo de 24 horas, con un selector de intervalo de tiempo configurable.
 - **AC6.** Una prueba de carga de 5 minutos con 10 clientes simultáneos se puede ver en el tablero en tiempo real durante la ejecución.
 - **AC7.** Las definiciones del tablero se almacenan como JSON en el repositorio en `monitoring/grafana/dashboards/`. Se aprovisionan automáticamente; no se requiere importación manual.
 - **AC8.** El archivo README documenta cómo acceder a Grafana (URL, credenciales predeterminadas) y cómo interpretar cada panel.
-

US-206 · Panel de rendimiento del motor HPC

Como analista de redes,

Quiero ver el rendimiento de procesamiento del motor HPC en un panel específico de Grafana,

para poder presentar datos concretos a Vault-Tec que demuestren el valor del procesamiento paralelo.

Criterios de aceptación:

- **AC1.** Grafana cuenta con un panel dedicado a HPC que muestra: gráficos procesados por minuto, latencia de procesamiento (P50, P95, P99) y la relación de aceleración en tiempo real frente a la línea de base de un solo subproceso.
- **AC2.** El panel se actualiza en tiempo real durante el procesamiento de gráficos.
- **AC3.** El informe de referencia de US-202 se menciona en la documentación del panel de control como evidencia histórica.
- **AC4.** Se activa una alerta si la latencia de procesamiento P99 supera el doble del valor de referencia de un solo subproceso durante más de 2 minutos consecutivos.
- **AC5.** Los tramos OTel del motor HPC son visibles en SigNoz y están vinculados a los tramos de origen del servidor v0.1 (mismo `trace_id`).
- **AC6.** El panel muestra la utilización de subprocesos: cuántos de los `OMP_NUM_THREADS` configurados están procesando activamente en cualquier momento.
- **AC7.** Un objetivo `make load-test-hpc` ejecuta una serie de solicitudes de análisis y genera un informe resumido que coincide con los datos del panel de Grafana.

5. Hito 3 — EOP v0.3 · API pública y entrega segura

Objetivo de la versión: Un ingeniero de integración puede consultar el estado de la red y activar operaciones a través de una API REST versionada. El equipo puede implementar nuevas versiones de EOP sin interrumpir el servicio mediante el lanzamiento canario y la implementación azul-verde.

Épico E8 — Pasar a API Gateway

US-301 · API REST de estado de nodo

Como ingeniero de integración,
quiero consultar el estado de todos los nodos de red a través de una API REST documentada,
para poder integrar EOP en sistemas externos sin conocer el protocolo de socket interno.

Criterios de aceptación:

- **AC1.** `GET /v1/nodes` devuelve una matriz JSON con todos los nodos registrados, incluyendo: `node_id`, `bunker_name`, `status`, `ip_address`, `last_seen_at`.
 - **AC2.** `GET /v1/nodes/{node_id}` devuelve todos los detalles de un nodo específico. Devuelve un `404` con un cuerpo de error estructurado si el nodo no existe.
 - **AC3.** Todas las respuestas utilizan `Content-Type: application/json`. Los cuerpos de respuesta siguen una estructura coherente: `{ "data": ..., "meta": { "timestamp": ... } }`.
 - **AC4.** La especificación OpenAPI 3.0 está disponible en `GET /v1/openapi.json` y es un entregable obligatorio de la versión v0.3.
 - **AC5.** La API se versiona bajo `/v1/`. Los cambios que rompen la compatibilidad requieren `/v2/`; esta política está documentada en el ADR.
 - **AC6.** La puerta de enlace delega al servidor C++ v0.1 utilizando el protocolo de socket existente. No reimplementa la lógica de negocio.
 - **AC7.** `GET /v1/health` devuelve `200 OK` con el estado del servicio. Este punto final se utiliza como sonda de disponibilidad de K8s.
 - **AC8.** Las pruebas de integración cubren: listar nodos (registro vacío), listar nodos (poblado), obtener nodo existente, obtener nodo inexistente (404) y comprobación de estado.
-

US-302 · API REST de análisis de grafos

Como ingeniero de integración,
Quiero iniciar un análisis de la topología de red a través de REST y recibir los resultados,
para poder ejecutar cálculos HPC desde sistemas externos sin conocer el protocolo interno.

Criterios de aceptación:

- **AC1.** `POST /v1/graph/analyze` acepta una carga útil JSON con el gráfico y los algoritmos solicitados. Devuelve resultados de forma sincrónica para gráficos con menos de 1000 nodos (en 5 s).
 - **AC2.** Para grafos $\geq 1K$ nodos, el punto final devuelve `202 Accepted` con un `job_id`. `GET /v1/graph/jobs/{job_id}` devuelve el estado del trabajo y los resultados cuando se completa.
 - **AC3.** Si el motor HPC está desconectado, el punto final devuelve el código `503 «Servicio no disponible»` junto con un mensaje descriptivo; el error interno no se muestra.
 - **AC4.** La puerta de enlace reenvía la solicitud al servidor v0.1 a través del protocolo de socket existente. No hay conexión directa entre la puerta de enlace y el motor.
 - **AC5.** Los resultados de los trabajos están disponibles durante al menos 5 minutos después de su finalización antes de ser eliminados.
 - **AC6.** `GET /v1/graph/jobs` devuelve la lista de trabajos recientes con su estado, hora de creación y hora de finalización.
 - **AC7.** Las cargas útiles de gráficos no válidas (campos faltantes, tipos incorrectos) devuelven un código `400 (Solicitud incorrecta)` con un error de validación estructurado que identifica el campo problemático.
 - **AC8.** El flujo asíncrono completo se verifica en una prueba de integración: `POST analyze` $\rightarrow$ `202` $\rightarrow$ `GET jobs/{id}` $\rightarrow$ `200` con resultados.
-

US-303 · Disyuntor para servicios externos

Como operador de Vault,

quiero que la puerta de enlace de la API tolere fallos en los servicios externos sin degradar el EOP, **de modo que** un fallo en la IA o en los servicios corporativos no provoque la caída de la plataforma.

Criterios de aceptación:

- **AC1.** La puerta de enlace implementa un interruptor de circuito por cada servicio externo (servicio AI Python, servicio Spring Boot corporativo).
 - **AC2.** El circuito se abre tras 5 errores consecutivos o una tasa de error $> 50\%$ en un intervalo de 10 segundos.
 - **AC3.** Mientras el circuito está abierto, la puerta de enlace devuelve una respuesta de respaldo predefinida (datos almacenados en caché o una respuesta degradada), no un error `500`.
 - **AC4.** El circuito pasa a estado `SEMIABIERTO` cada 30 segundos. Se cierra automáticamente si la solicitud de sondeo tiene éxito.
 - **AC5.** El estado de cada disyuntor (`ABIERTO`, `CERRADO`, `SEMIABIERTO`) se muestra como una métrica en Prometheus y es visible en el panel de control de SLO de Grafana.
 - **AC6.** La respuesta de respaldo está documentada en la especificación OpenAPI como la respuesta esperada cuando el circuito está abierto.
 - **AC7.** Una prueba de integración verifica lo siguiente: el servicio externo se cae $\rightarrow$ el circuito se abre tras alcanzar el umbral $\rightarrow$ se envía la respuesta de respaldo $\rightarrow$ el servicio se recupera $\rightarrow$ el circuito se cierra $\rightarrow$ se envía la respuesta normal.
 - **AC8.** La configuración del disyuntor (umbral, ventana, intervalo de semiabierto) se externaliza en un `ConfigMap`. Ningún cambio requiere una nueva implementación.
-

Epic E9 — Lanzamiento canario y despliegue azul-verde

Por qué existe esta épica en el producto:

EOP v1.0 debe mantener un tiempo de actividad superior al 99,9 % cuando se entregue a Vault-Tec. Una implementación ingenua (`kubectl set image`) tiene un intervalo de tiempo de inactividad. Los ingenieros aprenden los patrones de entrega que

eliminan ese lapso. El objetivo no es memorizar el concepto, sino ejecutarlo en vivo durante la demostración de la v0.3 con paneles que muestren la distribución real del tráfico.

US-304 · Lanzamiento canario de api-gateway

Como operador de Vault,

Quiero implementar una nueva versión de la puerta de enlace de la API de forma gradual, sin poner en riesgo a los usuarios actuales, **de modo que**, si la nueva versión presenta algún defecto, su impacto sea limitado y la reversión sea inmediata.

Criterios de aceptación:

- **AC1.** Coexisten dos implementaciones: `api-gateway-stable` (v1) y `api-gateway-canary` (v2), con un `Ingress` de `nginx` que divide el tráfico por peso.
 - **AC2.** El canario comienza con el 10 % del tráfico. El porcentaje se puede ajustar mediante una anotación (`nginx.ingress.kubernetes.io/canary-weight`) sin necesidad de volver a implementar.
 - **AC3.** SigNoz muestra métricas separadas para v1 y v2: tasa de error, latencia P99 y solicitudes por segundo.
 - **AC4.** Si la tasa de errores del canario supera el 1 % en un intervalo de 5 minutos, un procedimiento de reversión documentado establece la ponderación del canario en 0 en menos de 30 segundos.
 - **AC5.** La progresión del tráfico del 10 % al 50 % y al 100 % se ejecuta manualmente durante la demostración de la versión 0.3, con todos los paneles visibles.
 - **AC6.** Los clientes no experimentan errores durante ningún cambio en la división del tráfico (verificado con una prueba de carga simultánea que se ejecuta durante toda la demostración).
 - **AC7.** Ambas implementaciones comparten el mismo `servicio` de K8s. Solo cambia la ponderación de `Ingress` para desviar el tráfico.
 - **AC8.** La configuración de Canary está controlada por versiones. Un `make canary-rollout WEIGHT=10` El objetivo «target» y el objetivo «`make canary-rollback`» automatizan los cambios de ponderación.
-

US-305 · Implementación azul-verde sin tiempo de inactividad

Como operador de Vault,

quiero alternar entre dos versiones completas de la puerta de enlace de la API sin tiempo de inactividad, **para que** Vault-Tec no experimente ninguna interrupción del servicio durante los despliegues de EOP.

Criterios de aceptación:

- **AC1.** Existen dos implementaciones completas en paralelo: `azul` (producción actual) y `verde` (nueva versión).
- **AC2.** El selector `de servicio` de K8s apunta a `azul`. El cambio a `verde` se realiza actualizando la etiqueta del selector.
- **AC3.** Las pruebas de humo automatizadas se ejecutan en el estado «verde» antes del cambio. Si alguna prueba de humo falla, el cambio no se produce y el estado «azul» permanece activo.
- **AC4.** El cambio de servicio no produce ningún error `5xx` en los clientes activos (verificado con una prueba de carga continua ejecutada durante todo el cambio).
- **AC5.** La reversión (verde → azul) utiliza el mismo mecanismo de conmutación de selector y se completa en menos de 10 segundos.

- **AC6.** El conjunto de pruebas de humo se define en `tests/smoke/` y abarca: comprobación de estado, listado de nodos y una solicitud de análisis de gráfico de muestra.
 - **AC7.** Un objetivo de `conmutación azul-verde` ejecuta las pruebas de humo y, si estas se superan, realiza la conmutación del selector de forma atómica.
 - **AC8.** El ciclo completo azul-verde (implementación verde → pruebas de humo → conmutación → verificación → reversión) se muestra en vivo en la demostración de la versión 0.3 sin que se observen errores en el cliente.
-

Epic E10 — Registro centralizado

US-306 · Acceso unificado a registros en Loki

Como operador de Vault,

quiero buscar registros de cualquier servicio EOP a través de una interfaz unificada,

Para no tener que ejecutar «`kubectl logs`» en cada pod por separado a la hora de diagnosticar un problema.

Criterios de aceptación:

- **AC1.** Loki recopila registros de todos los pods de EOP. Grafana Explore permite filtrar por servicio, nivel de registro y `trace_id`.
 - **AC2.** Cada entrada de registro incluye un campo `trace_id` que coincide con el rastreo OTel de la solicitud que lo generó.
 - **AC3.** Dado un span en SigNoz, un operador puede navegar hasta las líneas exactas del registro correspondientes a esa solicitud en Grafana Explore utilizando el `trace_id`.
 - **AC4.** Los registros se conservan durante un mínimo de 7 días. Los registros no se pierden durante los reinicios de los pods.
 - **AC5.** La recopilación de registros es automática: no se necesitan cambios de configuración a nivel de aplicación cuando se inicia un nuevo pod.
 - **AC6.** Una prueba de integración verifica lo siguiente: una operación fallida genera un span con `result=ERROR` en SigNoz Y un registro de error correspondiente en Loki con el mismo `trace_id`.
 - **AC7.** Una sección del runbook describe cómo correlacionar una alerta de Grafana con los registros de solicitud específicos que la activaron.
 - **AC8.** El volumen de registros está limitado: la política de retención de Loki se almacena como código en `monitoring/loki/config.yaml` y se aplica automáticamente.
-

6. Hito 4 — EOP v1.0 · Integración en el borde y entrega del producto

Objetivo de lanzamiento: Los dispositivos de borde en los búnkeres de Vault-Tec envían telemetría al sistema central de manera confiable. EOP v1.0 es un producto entregable con un SLA demostrable. La demostración a Vault-Tec es la puerta de aceptación final.

ZephyrOS en ESP32 — Alcance técnico:

El ESP32 ejecuta ZephyrOS. El firmware implementa dos subprocesos de aplicación de ZephyrOS:

(1) un subproceso de sensor que toma muestras de datos cada segundo, y (2) un subproceso de cliente MQTT que publica en el broker con QoS 1. El objetivo pedagógico del RTOS es el modelo de tareas concurrentes en un entorno con recursos limitados (512 KB de RAM). Esto es precisamente

por lo que se utiliza MQTT y no gRPC: la sobrecarga de HTTP/2 es físicamente inviable en ese hardware. Los ingenieros deben ser capaces de explicar esta compensación en la defensa final.

Epic E11 — ZephyrOS en ESP32 + Editor MQTT

US-401 · Publicación confiable de telemetría desde un dispositivo periférico

Como dispositivo periférico,

quiero publicar telemetría del búnker al sistema central cada segundo de manera confiable, **para** que el equipo de operaciones de Vault-Tec pueda monitorear los búnkeres remotos en tiempo real.

Criterios de aceptación:

- **AC1.** El firmware de ZephyrOS cuenta con un subproceso de sensores que toma muestras `del node_id`, la temperatura simulada y `el estado` cada segundo.
 - **AC2.** Un hilo de cliente MQTT publica las muestras en el tema `eop/bunker/{node_id}/telemetry` con QoS 1.
 - **AC3.** La latencia de publicación desde el muestreo hasta la entrega al broker es < 100 ms en condiciones normales de conectividad.
 - **AC4.** La carga útil es JSON: `{ "node_id": "...", "timestamp": "...", "temperature": ..., "status": "..." }`.
 - **AC5.** Si no se puede contactar con el servidor de intermediación, el firmware no se bloquea ni entra en un bucle de reintentos infinito. Registra el error y vuelve a intentarlo con un retraso exponencial (1 s, 2 s, 4 s, 8 s, máximo 30 s).
 - **AC6.** El firmware arranca y comienza a publicar en un plazo de 10 segundos tras el encendido.
 - **AC7.** Los parámetros de conexión del broker MQTT (`host`, `puerto`, `prefijo de tema`) se pueden configurar a través de un archivo `prj.conf` de ZephyrOS; no hay valores codificados en el código fuente.
 - **AC8.** El comando `de shell` de ZephyrOS `eop status` muestra el estado actual de la conexión, la última marca de tiempo publicada y el total de mensajes enviados desde el arranque.
-

US-402 · Almacenamiento en búfer sin conexión y entrega garantizada

Como dispositivo periférico,

Quiero almacenar los mensajes en un búfer local cuando se pierda la conexión wifi y enviarlos una vez que se restablezca la conexión, **para que** no se pierda ningún dato de telemetría de los búnkeres durante las interrupciones temporales de la red.

Criterios de aceptación:

- **AC1.** Cuando se pierde la conexión WiFi, el firmware pone los mensajes en cola en un búfer de RAM local con capacidad para al menos 60 mensajes (60 segundos de datos).
- **AC2.** Al volver a conectarse, el firmware vacía el búfer antes de publicar nuevos mensajes, conservando el orden FIFO.
- **AC3.** Si el búfer está lleno, se descartan los mensajes más antiguos. Este comportamiento está documentado en el archivo README del firmware y en la salida del shell de ZephyrOS.
- **AC4.** Una prueba manual verifica lo siguiente: desactivar el WiFi durante 30 segundos → volver a activarlo → todos los mensajes almacenados en el búfer llegan al broker sin duplicados.
- **AC5.** La reconexión utiliza un retroceso exponencial: 1 s, 2 s, 4 s, 8 s, máximo 30 s entre intentos.

- **AC6.** El hilo del cliente MQTT publica un evento `RECONNECTED` en `eop/bunker/{node_id}/events` cada vez que se restablece la conectividad, con el número de mensajes almacenados en el búfer entregados.
 - **AC7.** La deduplicación de mensajes se implementa en el ingestor de borde (US-404) utilizando el campo `message_id` en la carga útil. El firmware del ESP32 asigna un `message_id` que aumenta monótonamente a cada mensaje.
 - **AC8.** Los eventos de desbordamiento de búfer (mensajes descartados debido a que el búfer está lleno) se registran como una métrica y se pueden ver en el panel de Grafana Edge.
-

US-403 · Resiliencia de subprocesos de ZephyrOS

Como operador de Vault,

quiero que el firmware del ESP32 se recupere de fallos de subprocesos individuales sin intervención manual, **de modo que** los dispositivos periféricos sean resilientes y no requieran acceso físico para reiniciarse.

Criterios de aceptación:

- **AC1.** Cada subproceso de ZephyrOS tiene un watchdog independiente. Si un subproceso no se registra en un plazo de 5 segundos, el watchdog lo reinicia.
 - **AC2.** Se mantiene un contador de reinicios por cada subproceso y se publica en `eop/bunker/{node_id}/health` en cada reinicio.
 - **AC3.** Un reinicio del hilo MQTT no interrumpe el hilo del sensor (y viceversa). Ambos hilos operan de forma independiente.
 - **AC4.** Si ZephyrOS detecta un error fatal del kernel, el dispositivo se reinicia automáticamente y reanuda su funcionamiento normal.
 - **AC5.** El LED del ESP32 parpadea con un patrón distintivo cuando un hilo se encuentra en estado de error (documentado en el archivo README del firmware como indicador visual de depuración).
 - **AC6.** Un fallo simulado en un subproceso (inyectado en una compilación de prueba mediante una opción de Kconfig) activa el watchdog y el contador de reinicios se incrementa correctamente.
 - **AC7.** Los eventos de reinicio de subprocesos se publican en el tema de estado de MQTT en un plazo de 10 segundos desde que se produce el reinicio.
-

Epic E12 — Ingestor de borde (Go)

US-404 · Ingesta y enrutamiento de eventos MQTT

Como operador de Vault,

quiero que los eventos de los dispositivos periféricos lleguen automáticamente al servidor EOP sin necesidad de integración manual, **de modo que** la telemetría periférica forme parte del estado de la red que gestiona EOP.

Criterios de aceptación:

- **AC1.** El ingestor periférico se suscribe a `eop/bunker/+telemetry` en el broker MQTT al iniciarse.
- **AC2.** Cada mensaje recibido se reenvía al servidor C++ v0.1 utilizando el protocolo de socket existente.

- **AC3.** El ingestor implementa idempotencia: los mensajes con un `message_id` duplicado (visto en los últimos 5 minutos) se confirman al broker, pero no se reenvían al servidor.
 - **AC4.** Un span de OTel cubre la ruta desde la recepción del mensaje MQTT hasta la entrega por socket al servidor v0.1. El `trace_id` se propaga en los metadatos del mensaje.
 - **AC5.** Si el servidor v0.1 está desconectado, el ingestor vuelve a intentar la entrega con un mecanismo de retroceso y registra cada intento.
Los mensajes no se descartan silenciosamente.
 - **AC6.** El ingestor se vuelve a conectar al broker MQTT automáticamente si se interrumpe la conexión, con retroceso exponencial.
 - **AC7.** `GET /v1/edge/status` en la puerta de enlace de la API devuelve el estado de conexión del ingestor, los mensajes recibidos y los mensajes reenviados en los últimos 60 segundos.
 - **AC8.** Una prueba de integración verifica lo siguiente: el simulador ESP32 publica 10 mensajes → el ingestor recibe los 10 → el servidor v0.1 registra las actualizaciones → `GET /v1/nodes` refleja el estado actualizado.
-

US-405 · Panel de control de latencia de Edge

Como operador de Vault,

Quiero ver la latencia de extremo a extremo desde el ESP32 hasta el servidor EOP en Grafana,
para poder verificar que el canal de telemetría en el borde cumple con su SLO.

Criterios de aceptación:

- **AC1.** El rastreo OTel de cada mensaje de borde incluye: la marca de tiempo de publicación (de la carga útil del ESP32), la marca de tiempo de recepción MQTT en el ingestor y la marca de tiempo de entrega del socket al servidor v0.1.
 - **AC2.** SigNoz muestra la latencia de extremo a extremo (ESP32 → servidor v0.1) para cada mensaje como un rastreo individual.
 - **AC3.** Grafana cuenta con un panel de borde dedicado que muestra: mensajes por segundo, latencia de extremo a extremo (P50, P99) y tasa de pérdida de mensajes (desbordamientos de búfer).
 - **AC4.** Se activa una alerta si la latencia de extremo a extremo (P99) supera los 500 ms durante más de un minuto consecutivo.
 - **AC5.** El panel de borde forma parte del tablero de entrega de Vault-Tec que se muestra en la demostración de la versión 1.0.
 - **AC6.** Un objetivo de `prueba de borde` ejecuta una carga sintética de 5 minutos desde un simulador ESP32 y genera un resumen que coincide con los datos del panel de Grafana.
 - **AC7.** El panel muestra el estado de conectividad del broker para cada dispositivo de borde conocido (conectado / desconectado / visto por última vez).
-

Epic E13 — Entrega del producto EOP v1.0

US-406 · Implementación de producción reproducible

Como operador de Vault,

quiero recibir EOP v1.0 como un producto implementable que se inicie con un solo comando,
para que Vault-Tec pueda operar la plataforma sin el apoyo continuo del equipo de Exodus Systems.

Criterios de aceptación:

- **AC1.** Un gráfico de Helm o una superposición de kustomize implementa la pila completa de EOP v1.0 con `kubectl apply`. No se requieren pasos manuales después de la aplicación.
- **AC2.** El archivo README del repositorio contiene instrucciones de implementación verificadas y probadas en un clúster de Kubernetes limpio.
- **AC3.** Toda la configuración de producción (puertos, períodos de retención, umbrales de SLO) se encuentra en un archivo de configuración específico, independiente del código de la aplicación.
- **AC4.** La pila completa alcanza el estado «Ready» en menos de 3 minutos desde la ejecución de «`kubectl apply`».
- **AC5.** Un archivo `RUNBOOK.md` documenta: cómo escalar servicios individuales, cómo realizar una reversión y cómo interpretar el tablero de SLO de Grafana.
- **AC6.** Un archivo `CHANGELOG.md` documenta todos los cambios desde la versión v0.1 hasta la v1.0, organizados por lanzamiento.
- **AC7.** Todos los ADR (desde la v0.1 hasta la v1.0) se encuentran en `/docs/adr/`, enlazados desde el README, y están actualizados.
- **AC8.** Un objetivo «`make smoke-test-prod`» se ejecuta en la pila implementada y verifica: la conectividad del servidor, el registro de nodos, el análisis de grafos, la ingesta de mensajes de aristas y el estado de la puerta de enlace de la API
— todo pasa.

US-407 · Demostración del cumplimiento del SLA

Como operador de Vault,

quiero recibir evidencia documentada de que EOP cumplió con el SLA de disponibilidad del 99,9 % durante el ciclo de pruebas,

para tener una prueba contractual de la fiabilidad de la plataforma.

Criterios de aceptación:

- **AC1.** El panel de control de SLO de Grafana muestra los datos de cumplimiento de los últimos 30 días con una granularidad de 1 hora.
- **AC2.** El margen de error en el momento de la entrega es > 0 (no se incumplió el SLA durante el ciclo de pruebas).
- **AC3.** Se incluye una captura de pantalla del panel de control o una exportación del período de pruebas como entregable de la versión 1.0.
- **AC4.** Durante la demostración en vivo, se elimina un pod con `kubectl delete pod`. El MTTR (tiempo de recuperación tras el fallo del pod → pod listo) es visible en Grafana y es < 200 ms.
- **AC5.** Durante la demostración en vivo, se ejecuta una versión canario de EOP v1.0 con los paneles de control tanto de la versión estable como de la versión canario visibles, mostrando la distribución real del tráfico.
- **AC6.** Durante la demostración en vivo, se muestra un flujo de mensajes ESP32 de extremo a extremo: el dispositivo publica → aparece el rastro de SigNoz → `GET /v1/nodes` refleja la actualización.
- **AC7.** La demostración es reproducible: un archivo `make demo-script` documenta la secuencia exacta de comandos ejecutados durante la presentación.
- **AC8.** Tras la demostración, el sistema sigue funcionando con normalidad. La demostración en sí no deja el clúster en un estado degradado.

7. Requisitos no funcionales

Los NFR son restricciones del sistema, no características opcionales. El incumplimiento de un NFR impide el lanzamiento del laboratorio correspondiente.

NFR-1: Rendimiento

Requisito	Medición	Umbral	Aplicable desde
Latencia de respuesta del servidor C++ (P99)	Duración del rastreo de OTel	< 200 ms	v0.1 →
Mínimo de clientes TCP simultáneos	Prueba de carga con N clientes simultáneos	≥ 10 sin degradación	v0.1
Aceleración HPC con 4 subprocesos (gráfico de 50 000 nodos)	Comparativa entre un solo subproceso y N subprocesos	≥ 2×	v0.2
Aumento de la velocidad de HPC con 8 subprocesos (grafo de 100 000 nodos)	Comparativa entre un solo hilo y N hilos	≥ 3×	v0.2
Latencia P99 de la API REST (de extremo a extremo)	Prometheus + Grafana	< 300 ms	v0.3 →
Latencia de extremo a extremo en el borde (ESP32 → servidor v0.1, P99)	Traza de OTel con span del ingestor	< 500 ms	v1.0

NFR-2: Fiabilidad

Requisito	Medición	Umbral	En vigor desde
Disponibilidad del servicio	Sonda de tiempo de actividad de Prometheus cada 30 s	≥ 99,9 % mensual	v0.2 →
MTTR en caso de fallo del pod	Grafana: muerte del pod → tiempo de preparación del pod	< 200 ms	v0.2 →
Apagado ordenado en SIGTERM	Código de salida 0. Sin errores de clientes activos.	100 % de los casos	v0.1 →
Cero pérdida de mensajes durante el almacenamiento en búfer sin conexión	WiFi desactivado durante 30 s → reconexión → comparación de recuentos	0 mensajes perdidos	v1.0
Tiempo de reversión de Canary	Cronometrado durante la demostración en vivo	< 30 s	v0.3 →
Conmutación azul-verde sin tiempo de inactividad	Recuento de errores 5xx durante la prueba de conmutación bajo carga	0 errores	v0.3 →

NFR-3: Calidad del código

Requisito	Herramienta de verificación	Gravedad
RAII para todos los recursos (sockets, fds, memoria, mutexes)	Revisión de código + AddressSanitizer	BLOQUEO
Sin fugas de memoria	AddressSanitizer <code>--leak-check=full</code> en CI	BLOQUEADO
Sin conflictos de datos	ThreadSanitizer en el conjunto de pruebas de concurrencia en CI	BLOQUEANTE
Las excepciones de C++ no cruzan los límites de los módulos	Revisión de código. Las interfaces públicas utilizan códigos de error.	BLOQUEO
Cobertura de pruebas ≥ 80 % en la lógica de negocio	<code>gcov</code> o <code>llvm-cov</code> en CI	Requerido
Clang-Tidy: sin advertencias críticas (propenso a errores, concurrencia)	<code>clang-tidy</code> en CI — fusión de bloques	Requerido
Imagen de Docker en tiempo de ejecución ≤ 50 MB	Imágenes de Docker verificación en CI	Requerido
Los manifiestos de K8s pasan la validación del esquema	<code>kubeval</code> o <code>kubeconform</code> en CI	Requerido

NFR-4: Operatividad

- Todos los servicios exponen un punto final de estado o una sonda de actividad de K8s que responde en menos de 100 ms.
 - Todos los registros están estructurados en JSON en stdout con los campos: `timestamp`, `level`, `service`, `trace_id`, `span_id`, `message`.
 - Cada operación relevante emite un span de OTel. Los spans son visibles en SigNoz en un plazo de 30 segundos.
 - La pila completa se inicia con un solo comando (`docker compose up` o `kubectrl apply`).
 - La implementación es reproducible: los manifiestos idénticos producen resultados idénticos en cualquier clúster de Kubernetes compatible.
 - Toda la configuración está externalizada. No hay valores específicos del entorno codificados de forma rígida en el código fuente ni en las imágenes de contenedor.
-

8. Negociación de Proto — Contrato entre escuadrones

El modelo multiescuadrón

Los cuatro equipos (Catalyst, Hyperion, Nexus, Orion) son equipos de ingeniería que desarrollan el **mismo producto**. Sus servicios deben interoperar. Cada equipo tiene total flexibilidad para definir la granularidad de sus propios servicios, pero el protocolo de mensajes que conecta esos servicios es un **contrato compartido** que todos los equipos negocian colectivamente.

Escuadrón	Propiedad del servicio	Dependencia del protocolo
Catalyst	Definido en el ADR de límites de servicio del equipo.	El protocolo de mensajes acordado en la reunión de negociación de protocolos.
Hyperion	Definido en el ADR de límites de servicio del escuadrón.	Ídem.
Nexus	Definido en el ADR de límites de servicio de la unidad.	Ídem.
Orion	Definido en el ADR de límites de servicio del escuadrón.	Ídem.

Reunión de negociación Proto — Hito obligatorio, Semana 2

Antes de escribir cualquier código de implementación, los cuatro equipos se reúnen para negociar el protocolo de mensajes. Esta reunión tiene entregables formales.

Cada equipo se prepara antes de la reunión:

1. Lista de operaciones que su servicio necesita exponer (verbos, cargas útiles, respuestas esperadas).
2. Lista de operaciones que necesitan consumir de los servicios de otros equipos.
3. Un formato de mensaje propuesto: estructura (prefijado por longitud, basado en delimitadores), tipos de campos, codificación (binaria, JSON, personalizada).
4. Borrador del documento «Límites del servicio» (ADR): qué incluye el servicio, qué no incluye y qué garantías ofrece.

La reunión da como resultado:

1. **Protocolo v1.0:** Tipos de mensajes, campos, estructura y codificación. Aprobado por los cuatro equipos.
2. **Tabla de responsabilidades:** qué equipo implementa cada lado de cada canal de comunicación.
3. **Criterios de compatibilidad:** qué constituye un cambio que rompe la compatibilidad frente a uno que no la rompe.
4. **Fecha de congelación del prototipo:** después de esta fecha, los cambios que rompen la compatibilidad requieren un control de versiones explícito y una sesión de renegociación.

Por qué existe este proceso:

En los sistemas de producción, un equipo que modifica el protocolo de comunicación sin coordinarse con los equipos dependientes provoca fallos en el sistema para todos los usuarios. La «Proto Negotiation» simula exactamente esa dinámica: cuatro equipos con incentivos independientes deben llegar a un acuerdo técnico, documentarlo formalmente y respetar ese acuerdo a lo largo de todo el proyecto. El diseño de protocolos de comunicación entre servicios y la gestión de su evolución son habilidades fundamentales en la ingeniería de sistemas distribuidos.

9. Contrato de ingeniería

Ciclo de entrega de 3 pasos

Ninguna característica entra en implementación sin pasar por Diseño. Ningún lanzamiento sale al mercado sin Estabilización.

%	Fase	Puerta de salida	Artefactos obligatorios
30	Diseño	ADR aprobados. Interfaces definidas. Protocolo negociado (v0.1).	Documentación de ADR en /docs/adr/ . Diagrama de secuencia del protocolo.
50 %	Implementación	Las pruebas pasan en CI. Sin fugas. Sin conflictos. Revisión de código aprobada.	Código + pruebas. CI en verde. PR con ≥ 1 revisión aprobada.
20	Estabilización	Las pruebas de integración pasan. Los SLO están en verde. Documentación actualizada.	Entrada README. CHANGELOG actualizada. Etiqueta de lanzamiento en git.

Definición de «Hecho»

Una historia se considera **«Hecha»** cuando se cumplen todas las condiciones siguientes:

1. El código se fusiona con [el principal](#) con CI completamente en verde.
2. Todos los criterios de aceptación son verificables mediante una prueba automatizada o una demostración reproducible.
3. Si la historia implica una decisión arquitectónica, el ADR correspondiente se documenta en [/docs/adr/](#).
4. Los cambios de la historia no introducen fugas de memoria ni conflictos de datos (verificado por los sanitizadores de CI).
5. El archivo README del servicio refleja el estado actual: cómo compilar, cómo ejecutar localmente y cómo ejecutar el conjunto de pruebas.

Puertas de lanzamiento

Ningún equipo avanza al siguiente Lab sin pasar la puerta del actual.

Puerta	Plazos	Criterio	Propietario
Congelación de prototipos	Fin de la semana 2	Protocolo v1.0 documentado y aprobado por los cuatro equipos.	PL + Todos los equipos
Lanzamiento de la v0.1	Fin de la semana 5	US-101 a US-110 Hecho. CI en verde. EOP v0.1 listo para implementación.	Escuadrón + PL
Lanzamiento de la versión v0.2	Fin de la semana 9	US-201 a US-206 Hecho. SLO Panel de control activo. K8s todo en verde.	Squad + PL
Lanzamiento de la versión v0.3	Fin de la semana 12	US-301 a US-306 completadas. Demostración del lanzamiento canario en vivo.	Squad + PL
EOP v1.0 — Final	Semana 16 — Demostración de Vault-Tec	US-401 a US-407 Hecho. SLA demostrable. Gráfico de timón desplegable.	Escuadrón + Panel de revisión

ADR

Arquitectura Registros de decisiones — Plataforma de operaciones Exodus (EOP)

Proyecto: Plataforma Operativa Exodus (EOP)

Cliente: División de Reconstrucción de Vault-Tec

Ingeniería: Exodus Systems Inc.

Documento: Índice ADR — PRD-EOP-2077-001

Estado: Documento en evolución — actualizado con cada lanzamiento de Lab

Cada ADR representa una decisión **vinculante** para los equipos responsables de los componentes afectados. Para revocar una decisión se requiere un nuevo ADR que sustituya explícitamente al anterior. Los ADR se almacenan en [/docs/adr/](#) en el repositorio y se enlazan desde el README.

Índice

ADR	Título	Estado	Lab
ADR-001	Mecanismo IPC — Sockets BSD sobre marcos RPC	Aceptado	v0.1
ADR-002	Modelo de concurrencia del servidor	Aceptado	v0.1
ADR-003	Diseño del protocolo de mensajes	Aceptado	v0.1
ADR-004	Límites del servicio (plantilla)	Aceptado	v0.1
ADR-005	Estrategia de observabilidad	Aceptado	v0.1
ADR-006	Contenedorización y adopción de Kubernetes	Aceptada	v0.1
ADR-007	Estrategia de paralelización — OpenMP	Aceptado	v0.2
ADR-008	Patrón de integración del motor HPC	Aceptado	v0.2
ADR-009	Definición y aplicación de los SLO	Aceptado	v0.2
ADR	Título	Estado	Laboratorio

ADR-010	API Gateway Idioma — Go	Aceptado	v0.3
ADR-011	Estrategia de implementación — Canary y Blue-Green	Aceptado	v0.3
ADR-012	Configuración del disyuntor	Aceptado	v0.3
ADR-013	Selección del protocolo de borde: MQTT sobre gRPC	Aceptado	v1.0
ADR-014	Arquitectura de subprocesos de ZephyrOS	Aceptado	v1.0

ADR-001: Mecanismo Mecanismo — BSD
 Sockets sobre marcos RPC

Fecha: Semana 1

Estado: Aceptado

Laboratorio: 1

Decisores: Todos los equipos + Instructor

Contexto

La plataforma EOP requiere comunicación entre procesos entre servicios distribuidos. Existen múltiples opciones en diferentes niveles de abstracción: sockets BSD/POSIX sin procesar, gRPC sobre HTTP/2, ZeroMQ o brokers de mensajes como RabbitMQ. La elección en esta capa determina el grado de control que tienen los ingenieros sobre el modelo de comunicación y afecta directamente a los objetivos de aprendizaje de la v0.1.

El objetivo principal de ingeniería de la versión 0.1 es que los estudiantes trabajen en la capa de transporte: gestionar descriptores de archivos, implementar la estructura de los mensajes, manejar lecturas y escrituras parciales, y razonar sobre la concurrencia en las conexiones de red. Estas competencias no pueden desarrollarse si un marco de RPC abstrae el socket.

Decisión

El servidor C++ utiliza sockets BSD/POSIX directamente. El servidor llama a `socket()`, `bind()`, `listen()`, `accept()`, `recv()` y `send()` sin ninguna biblioteca RPC o de mensajería intermedia. La biblioteca cliente C utiliza la misma API de sockets POSIX.

El protocolo de mensajes (tipos, encapsulación, serialización) es diseñado por cada equipo y documentado en ADR-003. El contrato de protocolo entre equipos se negocia colectivamente en la Reunión de Negociación de Proto.

Alternativas consideradas

Alternativa	Razón del rechazo
gRPC / Protobuf	Abstrae el socket por completo. Los ingenieros trabajarían en la capa RPC sin comprender la mecánica de transporte subyacente. No se cumpliría el objetivo de aprendizaje de la v0.1.
ZeroMQ	Sigue abstrayendo el framing y la gestión de conexiones. La misma objeción que con gRPC para el contexto de la v0.1.
RabbitMQ / Kafka	Introduce una dependencia del broker y desplaza el dominio del problema por completo. No es apropiado para un curso de fundamentos de sistemas distribuidos.
Sockets de dominio Unix	Limitado a un solo host. EOP requiere comunicación entre nodos a través de la red desde el inicio.

Consecuencias

Positivas:

- Los ingenieros desarrollan competencia directa con la API de sockets POSIX utilizada en sistemas de producción (bases de datos, servidores web, motores de juegos).
- Control total sobre el framing, el modelo de concurrencia y el manejo de errores: estos se convierten en decisiones de ingeniería explícitas.
- El protocolo diseñado en la v0.1 es propiedad de los equipos y está documentado en ADR, no generado por un marco de trabajo.

Negativas:

- Requiere más esfuerzo de implementación en comparación con el uso de gRPC. El franqueo, el manejo de lecturas parciales y la concurrencia deben implementarse desde cero.
- La evolución del protocolo requiere disciplina: la compatibilidad con versiones anteriores no la impone un compilador, sino que la impone el proceso de negociación de Proto y ADR-003.

Restricciones introducidas:

- Todo el encuadrado de mensajes debe manejar retornos parciales de `recv()`. El protocolo debe definir los límites de los mensajes de forma inequívoca.
- La gestión de recursos (descriptores de archivos, búferes) recae íntegramente en la aplicación. RAII es obligatorio según NFR-3.
- El modelo de concurrencia para manejar múltiples clientes debe elegirse explícitamente y documentarse en ADR-002.

ADR-002: Modelo de concurrencia del servidor

Fecha: Semanas 1–2 (por equipo, antes de la implementación)

Estado: Aceptado

Laboratorio: 1

Responsables: Cada equipo

Nota: Cada equipo documenta su propio modelo de concurrencia utilizando este ADR como plantilla. La decisión puede variar entre equipos. El requisito es que la elección se haga de forma explícita, se analicen sus ventajas e inconvenientes, y la implementación se ajuste al ADR.

Contexto

El servidor EOP C++ debe admitir un mínimo de 10 clientes simultáneos (NFR-1). Cuando un cliente envía un mensaje, el servidor debe procesarlo sin bloquear a otros clientes conectados. En el entorno POSIX hay tres modelos principales de concurrencia disponibles: un subproceso por conexión, un grupo de subprocesos y multiplexación de E/S (`select` / `poll` / `epoll`).

La elección afecta a: el consumo de memoria por cliente, el número máximo de conexiones simultáneas, la complejidad de la implementación y la exposición a primitivas de sincronización.

Decisión

[Cada equipo documenta aquí el modelo elegido.]

Las tres opciones y sus ventajas e inconvenientes son:

Opción A — Un subproceso por conexión

Se crea un hilo del sistema operativo por cada conexión de cliente aceptada. El hilo gestiona toda la comunicación de ese cliente y se cierra cuando se cierra la conexión.

Ventaja y desventaja	Detalle
Simplicidad	Cada hilo gestiona un cliente de forma secuencial. No se necesita sincronización explícita para el estado por cliente.
Ventajas e inconvenientes	Detalle
Memoria	Cada hilo consume memoria de pila (normalmente entre 2 y 8 MB). 100 clientes = 200–800 MB de pila.

Límite de escalabilidad	La sobrecarga de creación de subprocesos del sistema operativo se vuelve significativa cuando hay cientos de conexiones simultáneas.
Apto cuando	El número de clientes es limitado y bajo (< 100). Se prioriza la claridad de la implementación.

Opción B: grupo de subprocesos

Un grupo fijo de subprocesos de trabajo procesa las solicitudes de los clientes. Las conexiones aceptadas se ponen en cola y se asignan a los trabajadores disponibles.

Compensación	Detalle
Predecibilidad de la memoria	El número de subprocesos es fijo. El consumo de memoria está limitado independientemente del número de clientes.
Complejidad	Requiere una cola de trabajo segura para subprocesos. Sincronización entre el productor (bucle de aceptación) y los consumidores (trabajadores).
Escalabilidad	Mejor que un subproceso por conexión para un mayor número de clientes, pero el tamaño del grupo es un parámetro de ajuste.
Adecuado cuando	El número de clientes es variable o elevado, y es importante que el consumo de recursos sea predecible.

Opción C — Multiplexación de E/S (`epoll`)

Un solo hilo (o un pequeño número de hilos) supervisa múltiples descriptores de archivo simultáneamente utilizando `epoll`. El procesamiento está impulsado por eventos: se notifica al hilo cuando un descriptor de archivo está listo para leer o escribir.

Ventajas y desventajas	Detalle
Escalabilidad	Maneja miles de conexiones simultáneas con una sobrecarga mínima. Utilizado por nginx, Redis, Node.js.
Memoria	Sobrecarga por conexión casi nula más allá del fd y el estado a nivel de aplicación.
Ventajas y desventajas	Detalle
Complejidad	Máxima complejidad de implementación. Requiere máquinas de estados explícitas por conexión.
Adecuado cuando	Se requiere la máxima escalabilidad. El equipo de ingeniería tiene tiempo para implementarlo correctamente.

Consecuencias

El equipo documenta aquí las consecuencias de la opción elegida:

- Qué primitivas de sincronización se utilizan (mutexes, variables de condición, operaciones atómicas) y dónde.
 - Cuál es el estado compartido y cómo se mantienen sus invariantes.
 - Cómo se prueba el modelo bajo carga concurrente (véase US-102, AC-6, AC-8).
 - Cuál es el límite máximo práctico de clientes bajo el modelo elegido.
-

ADR-003: Diseño del protocolo de mensajes

Fecha: Semana 2 — Reunión preliminar de negociación

Estado: Aceptado

Laboratorio: 1

Responsables: Todos los equipos colectivamente

Nota: Los equipos completan los detalles específicos del protocolo (tipos de mensajes, nombres de campos, codificación) durante la reunión de negociación del prototipo. Este ADR define la **estructura de la decisión** —las propiedades que debe tener el protocolo— y sirve como plantilla que los equipos completan colectivamente.

Contexto

Los servicios EOP se comunican a través de TCP utilizando un protocolo personalizado. Para que el sistema funcione como un producto completo (y no como cuatro proyectos independientes), todos los equipos deben utilizar un protocolo que:

1. Defina los límites de los mensajes de forma inequívoca (enmarcado).
2. Identifique los tipos de mensajes y sus cargas útiles.
3. Puedo evolucionar sin afectar a los clientes existentes (estrategia de versionado).
4. Esté documentado con la precisión suficiente para que dos implementaciones independientes sean interoperables.

El protocolo es la decisión técnica más vinculante de todo el proyecto. Cada Lab posterior se basa en él.

Decisión

El protocolo debe especificar lo siguiente, decidido colectivamente en la Reunión de Negociación de Proto:

Enmarcado

El mecanismo mediante el cual el receptor sabe dónde termina un mensaje y dónde comienza el siguiente. Dos opciones:

Opción	Descripción	Ventajas e inconvenientes
Prefijo de longitud	Un encabezado de tamaño fijo (por ejemplo, un entero big-endian de 4 bytes) precede a cada mensaje y codifica la longitud de su carga útil.	Determinista. Admite cargas útiles binarias. Requiere dos llamadas a <code>recv()</code> por mensaje (encabezado y luego carga útil).
Delimitador	Una secuencia de bytes única (p. ej., <code>\n</code> , <code>\r\n</code> o un sentinelado personalizado) marca los límites del mensaje.	Más sencillo de implementar. Requiere escapar si el delimitador puede aparecer en la carga útil. No es adecuado para cargas útiles binarias.

Los equipos documentan su elección y justificación aquí.

Envoltorio del mensaje

Cada mensaje, independientemente de su tipo, debe contener:

Campo	Tipo	Descripción
<code>message_id</code>	uint32 o cadena	Identificador único de este mensaje. Se utiliza para la deduplicación y la correlación de solicitud-respuesta.
<code>message_type</code>	uint8 o cadena de enumeración	Identifica la operación (p. ej., <code>REGISTER</code> , <code>ACK</code> , <code>QUERY_NODE</code>).
<code>protocol_version</code>	uint8	Versión del protocolo. Permite el control de versiones en el futuro. El valor es <code>1</code> para la v0.1.
<code>longitud_de_la_carga</code>	uint32	Longitud de la carga útil en bytes. Cero para mensajes sin carga útil.
<code>carga</code>	bytes	Datos específicos del tipo de mensaje. La codificación se decide a continuación.

Codificación de la carga útil

Opción	Compromiso
JSON	Legible para humanos. Fácil de depurar con <code>netcat</code> o Wireshark. Mayor sobrecarga de análisis.
Binario personalizado	Compacto. Rápido de analizar. Requiere una especificación cuidadosa de los tamaños de los campos y el orden de los bytes.
Protobuf (sin gRPC)	Aplicación de esquema. Compacto. Requiere <code>la</code> . Aumenta la complejidad de la compilación.

Los equipos documentan su elección y justificación aquí.

Tipos de mensajes (conjunto mínimo requerido)

Tipo	Dirección	Descripción
REGISTRAR	Cliente → Servidor	Registrar un nodo con metadatos.
ACK	Servidor → Cliente	Confirmación con carga útil de resultado opcional.
ERROR	Servidor → Cliente	Respuesta de error con código y descripción legible para humanos.
QUERY_NODE	Cliente → Servidor	Consultar el estado de un nodo mediante <code>node_id</code> .
LIST_NODES	Cliente → Servidor	Solicitar el registro completo de nodos.
HEARTBEAT	Cliente → Servidor	Señal de mantenimiento de conexión del cliente al servidor.
ANALIZAR_GRÁFICO	Cliente → Servidor	Solicita análisis de gráficos (añadido en la extensión del protocolo v0.2).

Los equipos pueden agregar tipos de mensajes. Eliminar o modificar tipos existentes después de la congelación del protocolo (Proto Freeze) constituye un cambio de compatibilidad.

Política de versiones

- Cambios que añaden campos opcionales a mensajes existentes: **no rompen la compatibilidad**.
- Cambios que añaden nuevos tipos de mensajes: **no rompen la compatibilidad**.
- Cambios que eliminan campos, renombran campos o cambian tipos de campos: **rompen la compatibilidad** — requieren una nueva versión del protocolo y renegociación.

Consecuencias

- El protocolo se **congela al final de la Semana 2**. No se permiten cambios de compatibilidad tras la etapa de congelación del protocolo (Proto Freeze) sin una sesión de renegociación y un ADR actualizado.
- Todos los equipos implementan tanto el lado del servidor como la biblioteca del cliente para este mismo protocolo. La interoperabilidad se verifica mediante una prueba de integración que conecta un cliente del Equipo A con el servidor del Equipo B.
- La documentación del protocolo es un artefacto de primera clase. Cualquier ingeniero que se incorpore a un equipo a mitad de un proyecto debe ser capaz de implementar un cliente compatible a partir únicamente del ADR, sin necesidad de leer el código fuente.

ADR-004: Límites de servicio (Plantilla)

Fecha: Semanas 1–2 (por escuadrón)

Estado: Aceptado

Laboratorio: 1

Decisores: Equipo individual

Nota: Cada equipo redacta su propia instancia de este ADR. En él se documenta lo que hace su servicio, lo que explícitamente no hace y qué garantías ofrece a otros servicios que dependen de él. Esta es la aportación que cada equipo lleva a la reunión de negociación del prototipo.

Contexto

EOP está construido por cuatro equipos que trabajan en paralelo en el mismo producto. Sin límites explícitos de servicio, las responsabilidades se superponen, los puntos de integración no están definidos y la Reunión de Negociación Proto no puede producir un contrato de protocolo útil.

Cada equipo tiene total flexibilidad para definir la granularidad de sus servicios (un servicio, dos servicios o más). La restricción es que los límites deben documentarse para que otros equipos puedan razonar sobre en qué pueden y en qué no pueden depender.

Plantilla de decisión

Cada equipo documenta lo siguiente:

Servicios propios

Enumere cada servicio que posee el equipo. Para cada servicio:

Campo	Contenido
Nombre del servicio	Identificador único utilizado en manifiestos, registros y rastreos de K8s.
Responsabilidad	Qué hace este servicio. Un párrafo, específico.
Campo	Contenido
Operaciones expuestas	¿Qué tipos de mensajes maneja este servicio como servidor?

Operaciones consumidas	Qué tipos de mensajes envía este servicio a otros servicios como cliente.
Propiedad del Estado	De qué datos es fuente autorizada este servicio.
Garantías proporcionadas	En qué pueden confiar los solicitantes (p. ej., «responde en 200 ms bajo carga nominal», «el registro de nodos es consistente después de enviar el ACK»).

No responsabilidades explícitas

Lo que este servicio **no** hace. Esta sección es tan importante como la de responsabilidades. Evita la desviación del alcance y aclara los contratos de integración.

Dependencias

¿De qué servicios de otros equipos depende este servicio? ¿Qué supone sobre su comportamiento?

Consecuencias

- El ADR de límites de servicio es la información de entrada para la reunión de negociación Proto. Los equipos que llegan sin un borrador del ADR no pueden negociar de manera productiva.
- Después de la congelación de Proto, los servicios enumerados en este ADR son vinculantes. Agregar un nuevo servicio requiere una nueva entrada en el ADR y notificar a los equipos dependientes.
- El nombre del servicio definido aquí se utiliza de manera consistente en todos los artefactos: manifiestos de K8s, atributos de servicio de OTel, paneles de Grafana y etiquetas de registro.

ADR-005: Estrategia de observabilidad

Fecha: Semana 1

Estado: Aceptado

Laboratorio: 1

Decisores: Todos los equipos + Instructor

Contexto

EOP es un sistema distribuido. Cuando algo sale mal, la primera pregunta siempre es: «¿Qué pasó, dónde y cuándo?». Sin instrumentación, la única herramienta de diagnóstico son `los registros de kubectl` en pods individuales, lo cual no es escalable y no puede mostrar la causalidad más allá de los límites del servicio.

La observabilidad debe diseñarse en el sistema desde el principio, no añadirse más tarde. Adaptar el rastreo distribuido a un código base sin rastreo es significativamente más costoso que instrumentarlo desde el primer día.

Decisión

La pila de observabilidad para EOP es:

Capa	Herramienta	Introducida en	Propósito
Rastros distribuidos	SDK de OpenTelemetry + SigNoz	v0.1	Rastrear la causalidad más allá de los límites de los servicios. Respuesta: «¿Qué solicitud provocó este error?»
Métricas y SLO	Prometheus + Grafana	v0.2	Mide el estado del sistema a lo largo del tiempo. Respuesta: «¿Está el sistema cumpliendo su SLA en este momento?»
Registros centralizados	Loki + Grafana Explorar	v0.3	Corpus completo de registros consultable por servicio, nivel y <code>trace_id</code> . Respuesta: «¿Qué sucedió exactamente a nivel de línea?»

OpenTelemetry (OTel) — Estándar de instrumentación

Todos los servicios utilizan el SDK de OpenTelemetry independientemente del lenguaje (C++ o Go). Esto garantiza:

- Un único `trace_id` se propaga a través de todos los servicios, lo que permite la reconstrucción de trazas de extremo a extremo.
- Cambiar el backend (por ejemplo, de SigNoz a Jaeger) solo requiere un cambio de configuración, no un cambio de código.
- La instrumentación es coherente: la denominación de los spans, las claves de los atributos y el registro de errores siguen las convenciones semánticas de OTel.

Requisitos de span (mínimo por servicio)

Cada servicio debe generar un span para cada uno de los siguientes casos, como mínimo:

- Manejo de solicitudes entrantes (span del lado del servidor)
- Solicitud saliente a otro servicio (span del lado del cliente)
- Condiciones de error: los spans deben establecer `status = ERROR` e incluir el mensaje de error como un atributo

Requisitos de registro

Todos los registros deben estar estructurados en JSON en stdout con los siguientes campos:

```
JSON
{
  "timestamp": "2077-01-01T00:00:00.000Z",
  "level": "INFO",
  "service": "node-svc",
  "trace_id": "4bf92f3577b34da6...", "span_id":
  "00f067aa0ba902b7",
  "message": "Nodo registrado correctamente",
  "node_id": "bunker-42"
}
```

El campo `trace_id` debe coincidir con el rastreo OTel de la solicitud que generó el registro, lo que permite la correlación entre el registro y el rastreo a partir de la versión v0.1.

Alternativas consideradas

Alternativa	Motivo del rechazo
Marco de registro personalizado	No proporciona correlación de trazas distribuidas. Los registros de cada servicio están aislados.
Jaeger en lugar de SigNoz	SigNoz ofrece una experiencia más integrada de métricas y trazas en una única interfaz de usuario. El equipo ya está familiarizado con SigNoz.
ELK Stack (Elasticsearch + Logstash + Kibana)	Carga operativa significativamente mayor para implementar y mantener. Loki es más sencillo para un proyecto de curso y, al mismo tiempo, ofrece la funcionalidad básica necesaria.
Añadir observabilidad en la v0.3	Incorporar el rastreo distribuido en un sistema ya implementado es mucho más difícil. Los identificadores de rastreo deben estar presentes desde el primer servicio para permitir la correlación de extremo a extremo más adelante.

Consecuencias

- La instrumentación de OTel es un **requisito obligatorio** en la versión v0.1 (US-110). No es opcional ni se puede posponer.
- El OTel Collector se ejecuta como un contenedor independiente en `docker-compose.yml` a partir de la v0.1. Almacena en búfer la telemetría y la exporta a SigNoz. Si SigNoz no está disponible temporalmente, los tramos no se pierden.

- La correlación de `trace_id` entre SigNoz y Loki (disponible a partir de la v0.3) solo es posible porque el `trace_id` se incorporó en los registros a partir de la v0.1.
- Los ingenieros no deben registrar datos confidenciales (contraseñas, tokens, PII). La salida de los registros debe ser segura para mostrarla al equipo de operaciones.

ADR-006: Contenedorización y adopción de Kubernetes

Fecha: Semana 1

Estado: Aceptado

Laboratorio: 1

Responsables: Todos los equipos + Instructor

Contexto

EOP se entrega a Vault-Tec como un producto, no como código fuente. El cliente debe poder implementar el sistema de manera reproducible en su infraestructura. Vault-Tec opera un clúster de Kubernetes. La implementación debe poder ser gestionada por su equipo de operaciones sin un conocimiento profundo del código base de EOP.

Además, el plan de estudios del curso exige que los ingenieros operen cargas de trabajo nativas de la nube, incluyendo la gestión del ciclo de vida de los pods, las pruebas de estado y las implementaciones continuas.

Decisión

La contenedorización es obligatoria a partir de la versión 0.1. Kubernetes es el destino de implementación a partir de la versión 0.1.

Docker — Requisitos de compilación

- Se requieren **archivos Dockerfile de varias etapas** para todos los servicios en C++. La etapa de `compilación` contiene el conjunto de herramientas de compilación. La etapa de `ejecución` contiene únicamente el binario compilado y sus dependencias de ejecución.
- La imagen de tiempo de ejecución debe tener un tamaño ≤ 50 MB. Opciones de imagen base:
`debian:bookworm-slim,` `alpine:3.19,` `o`
`gcr.io/distroless/cc-debian12.`
- `ENTRYPOINT` debe utilizar el formato de matriz JSON: `["./eop-server"]`. El formato de shell (`ENTRYPOINT ./eop-server`) hace que el shell sea el PID 1, lo que no propaga `SIGTERM` a la aplicación.
- La aplicación debe gestionar `SIGTERM` con un apagado ordenado: dejar de aceptar nuevas conexiones, permitir que las conexiones activas se completen (máximo 15 s) y salir con el código `0`.

Kubernetes — Requisitos del manifiesto

Todos los servicios deben tener:

- Un `Deployment` con réplicas: 1 como mínimo. Las réplicas se pueden configurar mediante `ConfigMap`.
- Una prueba de disponibilidad de (socket TCP o HTTP GET a `/health`) con `initialDelaySeconds: 5, periodSeconds: 10, failureThreshold: 3`.
- Una sonda de disponibilidad (mismo mecanismo que la de actividad). El pod no se agrega a los puntos de conexión del Servicio hasta que la sonda de disponibilidad pase.
- `resources.requests` y `resources.limits` definidos tanto para la CPU como para la memoria.
- Un `ConfigMap` para toda la configuración del servicio. No hay valores específicos del entorno codificados de forma rígida en la imagen.

Desarrollo local

- `docker-compose.yml` proporciona la pila completa de desarrollo local: todos los servicios EOP + OTel Collector + SigNoz.
- El comando «`docker compose up`» debe poner toda la pila en un estado correcto en menos de 60 segundos.

Alternativas consideradas

Alternativa	Razón del rechazo
Solo Docker Compose (sin K8s)	No proporciona gestión del ciclo de vida de los pods, reinicios basados en el estado de salud ni las estrategias de implementación requeridas en los Laboratorios 3 y 4. El producto debe poder implementarse en K8s.
Introducir K8s en la v0.2	La versión canario (v0.3) y la ingestión de borde (v1.0) se basan en primitivas de K8s. Introducir K8s tarde complica la curva de aprendizaje en el momento equivocado. Comenzar en la v0.1 normaliza K8s como el entorno operativo desde el principio.
Máquinas virtuales	Mayor sobrecarga, iteración más lenta, no se ajusta a cómo se operan los sistemas distribuidos modernos.

Consecuencias

- Los ingenieros con experiencia solo en Docker Compose tendrán una curva de aprendizaje en la v0.1. Se proporciona una estructura básica: `Dockerfile` base, `docker-compose.yml` y manifiestos de K8s anotados con comentarios que explican cada campo.
- El clúster de K8s para el desarrollo local es `minikube` o `k3s`. La elección es por equipo y se documenta en el README del equipo.
- Todos los manifiestos de K8s son validados por `kubeval` o `kubeconform` en CI. Los manifiestos no válidos bloquean la fusión.

- El registro de imágenes está documentado en el README del equipo. La CI envía las imágenes etiquetadas a `main` en cada fusión.

ADR-007: Estrategia de paralelización — OpenMP

Fecha: Semanas 5–6
Estado: Aceptado
Laboratorio: 2
Decisores: Todos los equipos + Instructor

Contexto

La versión v0.2 introduce un requisito de computación de alto rendimiento (HPC): la plataforma EOP debe procesar grandes grafos de topología de red (hasta 100 000 nodos) con algoritmos paralelos. Los algoritmos de grafos requeridos son: camino más corto (Dijkstra), componentes conectados y análisis de centralidad.

Hay múltiples enfoques de paralelización disponibles en el ecosistema C++: OpenMP, subprocesos estándar de C++ (`std::thread`), Intel TBB, CUDA (GPU) y MPI (memoria distribuida).

Decisión

El motor de procesamiento HPC utiliza OpenMP para la paralelización.

Se añaden directivas OpenMP a las implementaciones de los algoritmos de grafos. El número de subprocesos se controla mediante la variable de entorno `OMP_NUM_THREADS`.

```
C/C++

// Ejemplo: bucle externo paralelizado con OpenMP #pragma
                                omp          parallel          for          schedule(dynamic)
num_threads(omp_get_max_threads()) for (int i
= 0; i < num_nodes; ++i) {
    process_node(graph, i, results);
}
```

Alternativas consideradas

Alternativa	Motivo del rechazo
Paralelismo manual de <code>std::thread</code>	Requiere gestión manual del grupo de subprocesos, robo de trabajo y sincronización. Mayor complejidad de implementación para el mismo resultado. Adecuado para requisitos de programación personalizados — no es necesario aquí.
Alternativa	Motivo del rechazo

Intel TBB (Threading Building Blocks)	Más potente que OpenMP para el paralelismo irregular. Mayor sobrecarga de dependencia. OpenMP está disponible tanto en GCC como en Clang sin instalación adicional.
CUDA (paralelismo de GPU)	Requiere disponibilidad de hardware de GPU en todas las máquinas de desarrollo. No es viable para un proyecto de curso.
MPI (memoria distribuida)	Apto para clústeres HPC de múltiples nodos. EOP v0.2 se enfoca en el paralelismo de memoria compartida dentro de un solo proceso. MPI sería la opción correcta si el motor abarcara múltiples pods de K8s.

Consecuencias

- OpenMP es compatible tanto con GCC (`-fopenmp`) como con Clang (`-fopenmp`). El archivo CMakeLists.txt debe incluir `find_package(OpenMP REQUIRED)` y enlazarse con `OpenMP::OpenMP_CXX`.
- El número de subprocesos se puede configurar en tiempo de ejecución mediante `OMP_NUM_THREADS`. La implementación de K8s lo establece a través de una entrada de ConfigMap. Cambiar el número de subprocesos no requiere una nueva implementación.
- El informe de referencia (US-202) debe medir la aceleración con 1, 2, 4 y 8 subprocesos. Debe incluir un análisis de la Ley de Amdahl: calcular la fracción en serie y predecir la aceleración máxima teórica.
- Las carreras de datos son el riesgo principal. ThreadSanitizer debe reportar cero errores con cualquier número de subprocesos de 1 a `$(nproc)`. Esta es una puerta de CI obligatoria.
- Se deben utilizar cláusulas de reducción de OpenMP cuando sea aplicable para evitar la sincronización manual de las operaciones de acumulación.

ADR-008: Patrón de integración del motor HPC

Fecha: Semana 6

Estado: Aceptado

Laboratorio: 2

Participantes: Todos los equipos + Instructor

Contexto

El motor HPC introducido en la v0.2 podría diseñarse como un proceso independiente con su propia interfaz de cliente, o como un componente integrado de la capa de comunicación EOP existente. Esta decisión determina si la v0.2 se basa en la v0.1 o la reemplaza.

Decisión

El motor HPC se registra como un nodo EOP en el servidor v0.1 y recibe trabajo a través del protocolo de socket existente.

El patrón de integración es:

1. El motor HPC se inicia y envía un mensaje `REGISTER` al servidor v0.1 (igual que cualquier cliente EOP).
2. Cuando un cliente C envía un mensaje `ANALYZE_GRAPH` al servidor v0.1, el servidor envía la solicitud al nodo del motor HPC registrado.
3. El motor procesa el gráfico y envía el resultado al servidor.
4. El servidor reenvía el resultado al cliente de origen.

El tipo de mensaje `ANALYZE_GRAPH` se agrega al protocolo como una extensión no rompedora (según la política de versiones de ADR-003). Su definición es un apéndice de ADR-003.

Alternativas consideradas

Alternativa	Razón del rechazo
Proceso HPC independiente con su propio servidor de sockets	Rompe la continuidad del producto. El cliente necesitaría conocer un punto final independiente. El servidor v0.1 no participaría en el flujo, lo que rompería el hilo de continuidad.
Motor HPC como biblioteca vinculada al servidor v0.1	Crea un monolito. El motor no se puede escalar de forma independiente. Elimina el patrón de microservicios.
gRPC para la comunicación del motor	Inconsistente con ADR-001. Introduce un mecanismo de comunicación diferente para un subconjunto de servicios sin una justificación sólida.

Consecuencias

- La continuidad se impone arquitectónicamente: el motor HPC no puede demostrarse sin que el servidor v0.1 esté en ejecución. Las pruebas de integración de extremo a extremo deben recorrer la ruta completa.
 - Los spans de OTel deben cubrir toda la cadena de despacho: solicitud del cliente → despacho del servidor → procesamiento del motor → entrega del resultado. Todos los spans deben compartir el mismo `trace_id`.
 - Si el HPC motor está fuera de línea, el v0.1 servidor debe devolver `ERROR_SERVICE_UNAVAILABLE` al cliente. El servidor no debe bloquearse ni quedarse colgado esperando un motor no disponible.
 - El motor puede ejecutarse como una `tarea` de K8s (procesamiento por lotes de una sola ejecución) o como un (disponibilidad persistente). Ambos modos son compatibles y están documentados.
-

ADR-009: Definición y aplicación de SLO

Fecha: Semana 6

Estado: Aceptado

Laboratorio: 2

Responsables: Todos los equipos + Instructor

Contexto

EOP se entrega a Vault-Tec con una garantía contractual de disponibilidad del 99,9 % de tiempo de actividad mensual. Este SLA debe ser medible, no solo una afirmación. La versión v0.2 es el punto en el que el sistema pasa a Kubernetes y obtiene la infraestructura necesaria para medir y hacer cumplir los objetivos de nivel de servicio.

Un SLO sin un mecanismo de medición es una esperanza, no un compromiso.

Decisión

Los SLO se definen como código y se hacen cumplir a través de la pila Prometheus + Grafana. El incumplimiento de un SLO activa una alerta automatizada. El presupuesto de errores es visible en todo momento.

Definiciones de SLI (indicadores de nivel de servicio)

Los SLI son las métricas brutas de las que se derivan los SLO:

SLI	Método de medición	Recopilado por
Tasa de éxito de las solicitudes	Proporción de respuestas que no son 5xx respecto al total de solicitudes	Prometheus contador por código de respuesta
Latencia de la solicitud	Duración de las solicitudes en percentiles especificados	OTel histograma exportado a Prometheus
Disponibilidad del pod	Binario: ¿el pod está sirviendo tráfico?	Prometheus Métrica de estado desde Kubernetes
Latencia de procesamiento HPC	Duración de gráfico solicitudes de análisis de gráficos	OTel intervalo exportadas a Prometheus

Objetivos SLO

SLO	Objetivo	Ventana de medición	Error esto (mensual) Presupu
Disponibilidad del servicio	≥ 99,9 %	30 días consecutivos	43,8 minutos
Latencia del servidor C++ P99	< 200 ms	Promedio móvil de 5 minutos	0,1 % de las solicitudes
Latencia P99 de la API REST	< 300 ms	Promedio móvil de 5 minutos	0,1 % de las solicitudes
SLO	Objetivo	Ventana de medición	Error esto (mensual) Presupu

Latencia de extremo a extremo P99	< 500 ms	Promedio móvil de 5 minutos	0,1 % de las solicitudes
-----------------------------------	----------	-----------------------------	--------------------------

Política de presupuesto de error

- **Presupuesto de error > 50 %:** Operaciones normales. Las nuevas funciones y las implementaciones continúan según lo planeado.
- **Presupuesto de error del 10 % al 50 %:** precaución. Las nuevas implementaciones requieren una revisión adicional.
El enfoque se centra en las mejoras de confiabilidad.
- **Presupuesto de error < 10 %:** Congelación. No se implementarán nuevas funciones hasta que el presupuesto de error se recupere por encima del 10 %.

Alternativas consideradas

Alternativa	Motivo del rechazo
Monitorización supervisión(revisar los paneles de control periódicamente)	No ofrece alertas automatizadas. Es necesario que una persona esté observando el tablero para detectar una violación del SLO.
SLO definido únicamente en el documento contractual	Un SLO que no se mide no es un SLO. El cliente no puede verificar el cumplimiento.
Prometheus sin Grafana	Prometheus proporciona los datos. Grafana proporciona los paneles necesarios para la demostración de entrega de Vault-Tec (US-407). Ambos son necesarios.

Consecuencias

- Las definiciones de los paneles de Grafana se almacenan como JSON en el repositorio ([monitoring/grafana/dashboards/](#)). Se aprovisionan automáticamente; no se requiere ninguna importación manual después de la implementación.
- Todos los paneles de Grafana y las reglas de alerta están controlados por versiones. Los cambios en las definiciones de los paneles pasan por el proceso estándar de revisión de PR.
- El panel de presupuesto de errores es visible en el panel de demostración de Vault-Tec durante la revisión final de la v1.0. Un presupuesto de errores positivo en el momento de la entrega es un criterio de aceptación obligatorio (US-407, AC-2).

ADR-010: Lenguaje de API Gateway — Go

Fecha: Semana 9

Estado: Aceptado

Laboratorio: 3

Decisores: Todos los equipos + Instructor

Contexto

La v0.3 introduce una API REST pública que expone las capacidades de EOP a los consumidores externos. Este componente se sitúa entre el mundo externo y los servicios C++ creados en los Laboratorios 1 y 2. Se requiere una decisión sobre el lenguaje para este componente.

Las restricciones son: el componente debe integrarse con los servicios existentes basados en sockets C++, implementar patrones de resiliencia (disyuntor, reintentos, tiempos de espera) y exponer una API REST limpia con una especificación OpenAPI 3.0.

Decisión

La puerta de enlace API está implementada en Go.

Go se utiliza exclusivamente para los servicios `api-gateway` y `edge-ingestor`. C++ sigue siendo el lenguaje de implementación para todos los demás servicios de EOP.

Justificación

Criterio	Go	C++
Ecosistema HTTP/REST	De primera clase. <code>net/http</code> , <code>chi</code> , <code>gin</code> y <code>fiber</code> son aptos para producción.	Posible (Crow, Beast, Drogon), pero no idiomático. Mayor costo de configuración.
Modelo de concurrencia	Las goroutines y los canales son nativos. El manejo correcto de HTTP concurrente es sencillo.	Requiere una gestión explícita de subprocesos. Más código repetitivo para el mismo resultado.
Integración con C++	El cliente de socket TCP en Go son 20-30 líneas de código de la biblioteca estándar. CGO está disponible, pero no es necesario dado el protocolo de socket.	N/A
Compilación y velocidad de iteración	Rápida. Compilaciones para un tamaño de servicio típico.	Compilación más lenta. Mayor velocidad de iteración durante el desarrollo.
Implementación	Único binario binario. Imagen Docker mínima.	Binario estático único. Comparable.
Objetivo de aprendizaje	Los ingenieros aprenden un segundo lenguaje de sistemas con una semántica de concurrencia diferente. El contraste con C++ es valioso desde el punto de vista pedagógico.	No es un lenguaje nuevo. No cumple con el objetivo curricular de integración multilingüe.

Consecuencias

- La puerta de enlace Go se comunica con el servidor C++ v0.1 utilizando el protocolo de socket TCP existente definido en ADR-003. Sin CGO, sin gRPC. Un Go `net.Conn` es un cliente de socket TCP; se aplica el mismo protocolo.
 - La especificación OpenAPI 3.0 es un entregable obligatorio (US-301, AC-4). Se genera a partir de anotaciones de código o se mantiene como un archivo YAML/JSON independiente. Ambos enfoques son aceptables; la elección se documenta en el ADR del equipo.
 - El servidor HTTP de Go gestiona la concurrencia de forma nativa. No se necesita una gestión explícita de subprocesos para la puerta de enlace.
 - La puerta de enlace es un binario independiente con su propio `Dockerfile`, implementación de K8s y entrada de servicio en `docker-compose.yml`.
-

ADR-011: Estrategia de implementación — Canario y azul-verde

Fecha: Semana 9

Estado: Aceptado

Laboratorio: 3

Decisores: Todos los equipos + Instructor

Contexto

EOP v1.0 debe mantener una disponibilidad $\geq 99,9\%$ cuando se entregue a Vault-Tec. Cada implementación de una nueva versión conlleva el riesgo de introducir un defecto. Dos patrones de implementación eliminan o reducen significativamente el tiempo de inactividad asociado a las implementaciones: el lanzamiento canario y la implementación azul-verde.

Entender cuándo utilizar cada patrón es una competencia operativa fundamental. El requisito de la v0.3 no es memorizar las definiciones, sino ejecutar ambos patrones en implementaciones en vivo con paneles de control observables.

Decisión

Tanto el lanzamiento canario como el despliegue azul-verde se implementan en la v0.3 para el servicio `api-gateway`. Cada uno se utiliza en su contexto apropiado.

Lanzamiento canario: utilízelo cuando

La nueva versión presenta incertidumbres. El equipo quiere exponer un pequeño porcentaje del tráfico real a la nueva versión y observar su comportamiento antes de comprometerse con un lanzamiento completo.

Implementación:

- Dos implementaciones de K8s: `api-gateway-stable` (v1) y `api-gateway-canary` (v2).

- Un K8s `Ingress` (nginx) con anotaciones de peso canary que divide el tráfico entre las dos implementaciones.
- Progresión del tráfico: 10 % → 50 % → 100 %, ejecutada manualmente con observación del panel en cada paso.
- Revertido: establecer el peso canary en 0. La implementación estable absorbe el 100 % del tráfico.

Ninguna

```
# Anotación de Canary Ingress
nginx.ingress.kubernetes.io/canary: "true"
nginx.ingress.kubernetes.io/canary-weight: "10"
```

Implementación azul-verde: se utiliza cuando

La nueva versión ha superado todas las pruebas y el equipo desea ejecutar un cambio sin tiempo de inactividad. El patrón también se utiliza cuando la velocidad de reversión es la principal preocupación.

Implementación:

- Dos `implementaciones` completas de K8s: `api-gateway-blue` (producción actual) y `api-gateway-green` (nueva versión).
- Un `servicio` de K8s cuyo `selector` apunta a blue o a green.
- Pruebas de humo automatizadas ejecutadas en green antes de cambiar el selector.
- Cambio: actualizar el selector `del servicio de la versión: blue a la versión: green`. K8s actualiza la lista de puntos finales de forma atómica.
- Reversión: revertir el selector. No se requiere reimplementación.

Cuándo usar cada uno

Escenario	Patrón
Alta incertidumbre sobre el nuevo comportamiento de la versión bajo carga de producción	Canario
Cambios que son difíciles de probar de forma sintética (por ejemplo, impulsados por el comportamiento del usuario)	Canary
Cambios que han superado la cobertura total de pruebas y las pruebas de fumigación	Azul-Verde
La velocidad de reversión es la principal preocupación	Azul-Verde
Infraestructura de la infraestructura de ejecutar dos entornos completos es aceptable	Azul-Verde

Consecuencias

- Los ingenieros deben ser capaces de explicar las ventajas y desventajas de los dos patrones durante la demostración de la versión 0.3 y la defensa final.
- Las configuraciones canario y azul-verde están controladas por versiones. `make canary-rollout WEIGHT=10` y `make blue-green-switch` automatizan los mecanismos.
- SigNoz debe mostrar métricas separadas (tasa de error, latencia P99) para cada versión de implementación durante un despliegue canario. Esto requiere etiquetas de versión en los atributos de recursos de OTel.
- Se debe ejecutar una prueba de carga continua tanto durante la progresión canario como durante el cambio azul-verde. La ausencia de errores `5xx` durante cualquiera de las operaciones es un criterio de aceptación obligatorio.

ADR-012: Configuración del disyuntor

Fecha: Semana 9

Estado: Aceptado

Laboratorio: 3

Decisores: Todos los equipos + Instructor

Contexto

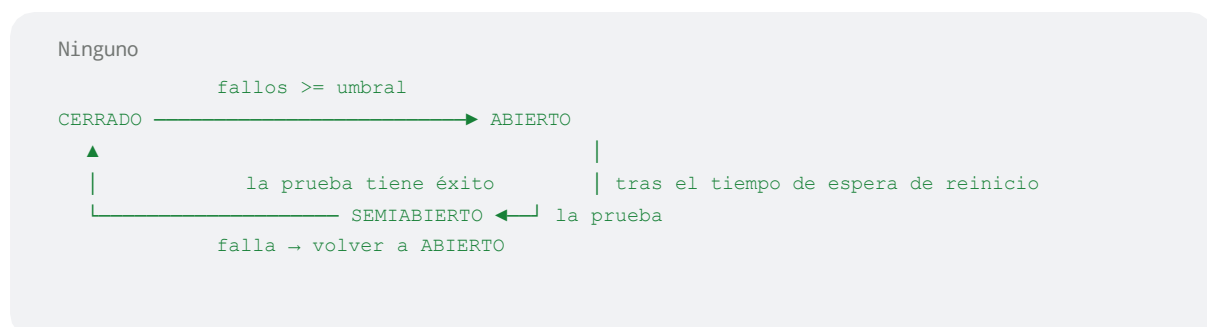
La puerta de enlace de la API de EOP se integra con dos servicios externos: un servicio de IA (Python) y un servicio corporativo (Spring Boot), ambos proporcionados por Vault-Tec. Estos servicios tienen características de implementación y confiabilidad independientes. Un fallo en cualquiera de los servicios externos no debe propagarse a EOP.

Sin un disyuntor, un servicio externo lento o defectuoso provoca que se acumulen todas las solicitudes que dependen de él, consumiendo goroutines y, finalmente, agotando los recursos de la puerta de enlace: un fallo en cadena.

Decisión

La puerta de enlace de la API implementa un disyuntor por cada dependencia de servicio externo.

Máquina de estados



Parámetros de configuración (externalizados en ConfigMap)

Parámetro	Predeterminado	Descripción
<code>failure_threshold</code>	5	Número de fallos consecutivos antes de que se abra el circuito.
<code>error_rate_threshold</code>	50	Tasa de error dentro de la ventana que activa la apertura.
<code>duración_de_la_ventana</code>	10 s	Tiempo ventana durante error.
<code>tiempo_de_reinicio</code>	30 s	Tiempo que el circuito permanece ABIERTO antes de pasar a SEMIABIERTO.
<code>half_open_max_calls</code>	1	Número de llamadas de sondeo permitidas en estado SEMI-ABIERTO.

Comportamiento de respaldo

Cada llamada externa protegida debe definir una respuesta de respaldo. El respaldo se devuelve cuando el circuito está en estado OPEN. Respaldos aceptables:

- Respuesta almacenada en caché de la última llamada realizada con éxito (con un indicador de antigüedad).
- Un servicio «servicio «respuesta degradada» con con HTTP 503 status y un encabezado `Retry-After`.
- Omisión de los datos externos de la respuesta (si el campo es opcional).

No es aceptable devolver un HTTP 500 cuando el circuito está abierto. El propósito del disyuntor es proteger al cliente de la propagación de errores internos.

Observabilidad

Las transiciones de estado del disyuntor deben emitirse como:

- Un evento de span de OTel en el span de la solicitud que desencadenó el cambio de estado.
- A Prometheus :
`eop_circuit_breaker_state_transitions_total{service, from_state, to_state}`.
- Un panel de Grafana que muestra el estado actual de cada disyuntor.

Alternativas consideradas

Alternativa	Razón por la que se rechazó
Solo tiempo de espera (sin disyuntor)	Los tiempos de espera evitan solicitudes que se cuelguen, pero no impiden que la puerta de enlace
Motivo	Motivo del rechazo

	intentar repetidamente un servicio que falla. En caso de fallo prolongado, la puerta de enlace sigue consumiendo recursos en cada tiempo de espera.
Reintentar solo con retroceso	Los reintentos aumentan la carga en un servicio que ya está fallando. Sin un disyuntor, los reintentos pueden amplificar el fallo.
Descartar inmediatamente las solicitudes con 503	No intenta la recuperación ni proporciona respuestas en caché o degradadas. Experiencia de usuario deficiente.

Consecuencias

- La configuración del disyuntor se encuentra en un `ConfigMap`. Cambiar los umbrales no requiere una nueva implementación.
- La respuesta de respaldo para cada servicio externo está documentada en la especificación OpenAPI como un posible esquema de respuesta.
- Los ingenieros deben demostrar el comportamiento del interruptor automático en la demostración v0.3: detener el servicio externo → observar la apertura del circuito → verificar la respuesta de respaldo → reiniciar el servicio externo → verificar el cierre del circuito.

ADR-013: Selección del protocolo de borde — MQTT sobre gRPC

Fecha: Semana 12

Estado: Aceptado

Laboratorio: 4

Decisores: Todos los equipos + Instructor

Contexto

La versión v1.0 introduce dispositivos periféricos (ESP32 con ZephyrOS) que deben enviar telemetría al sistema central EOP. Se debe seleccionar un protocolo de comunicación que funcione bajo las siguientes restricciones:

- **Hardware:** ESP32 con ~520 KB de SRAM y ~4 MB de memoria flash.
- **Conectividad:** WiFi con conectividad intermitente. El dispositivo debe soportar las desconexiones y volver a conectarse automáticamente.
- **ZephyrOS:** El cliente del protocolo debe tener una implementación compatible con ZephyrOS.
- **Semántica de los mensajes:** La telemetría se publica periódicamente. El dispositivo periférico no necesita esperar una respuesta para continuar su funcionamiento.

Esta decisión también es un objetivo pedagógico fundamental: los ingenieros deben comprender por qué las elecciones de protocolo realizadas para servicios con abundantes recursos (gRPC, sockets TCP) no siempre son adecuadas para entornos con restricciones.

Decisión

Los dispositivos periféricos utilizan MQTT sobre TCP para comunicarse con el sistema central EOP.

El broker MQTT se ejecuta como una implementación de K8s en el clúster EOP. Los dispositivos periféricos se conectan al broker y publican en el tema `eop/bunker/{node_id}/telemetry` con QoS 1.

Justificación: ¿por qué no gRPC?

Restricción	gRPC	MQTT
Sobrecarga del protocolo	Compresión de encabezados HTTP/2 (HPACK), negociación TLS, multiplexación de tramas. Memoria mínima de referencia: ~150–300 KB para una pila HTTP/2 compatible con TLS.	Encabezado fijo de 2 bytes por mensaje. Implementación mínima del cliente: ~20–40 KB.
RAM de ESP32 (520 KB en total)	Insuficiente tras tener en cuenta el kernel de ZephyrOS, la pila WiFi y la aplicación. Una implementación mínima de gRPC + TLS no cabe en el montón disponible.	Cabe sin problemas. ZephyrOS cuenta con un cliente MQTT de primera clase en su subsistema de redes.
Modelo de conectividad	Diseñado para conexiones estables. es es a nivel de aplicación y detallada.	Manejo de reconexión integrado. Los niveles de QoS definen las garantías de entrega de manera explícita.
Garantía de entrega de mensajes	La aplicación debe implementar la lógica de acuse de recibo sobre los flujos HTTP/2.	QoS 1 proporciona entrega «al menos una vez» de forma nativa. El broker persiste los mensajes para clientes sin conexión (clean session = false).
Compatibilidad con ZephyrOS	No nativo gRPC en en la pila de red de ZephyrOS.	<code>mqtt_client</code> es un módulo de subsistema Zephyr de primera clase.

Nivel de calidad de servicio (QoS) de MQTT

Se selecciona **QoS 1 (entrega al menos una vez)**:

- El broker confirma cada mensaje publicado con un `PUBACK`.
- Si el dispositivo no recibe el `PUBACK` (por ejemplo, debido a una desconexión), lo retransmite.
- Es posible que se produzcan mensajes duplicados. El `ingestor periférico` implementa la deduplicación utilizando el campo `message_id` (ADR-014, US-402 AC-7).

Se rechazó QoS 0 (fire-and-forget): la pérdida de telemetría durante las interrupciones no es aceptable según US-402.

Se rechazó QoS 2 (exactamente una vez): el protocolo de enlace de dos fases duplica la sobrecarga de mensajes, lo cual es excesivo para intervalos de telemetría de 1 segundo.

Consecuencias

- El broker MQTT (`mosquitto` o `hivemq`) se ejecuta como una implementación de K8s con un `PersistentVolume` para la retención de mensajes. Forma parte de la entrega del producto EOP (US-406).
- El `edge-ingestor` (Go) se suscribe al broker y reenvía los mensajes al servidor v0.1 C++ a través del protocolo de socket existente.
- Los ingenieros deben ser capaces de explicar la compensación entre gRPC y MQTT durante la defensa final. La pregunta «¿por qué no usaste gRPC en el ESP32?» debe tener una respuesta específica y cuantitativa que haga referencia a las limitaciones de memoria.
- La conexión MQTT utiliza TLS en la configuración de producción. La pila TLS de ZephyrOS (`mbedTLS`) está disponible y configurada en `prj.conf`.

ADR-014: Arquitectura de subprocesos de ZephyrOS

Fecha: Semana 12

Estado: Aceptado

Laboratorio: 4

Decisores: Todos los equipos + Instructor

Contexto

El firmware del ESP32 debe realizar dos actividades simultáneas: muestrear datos del sensor a un intervalo fijo y publicar esos datos a través de MQTT. Estas actividades tienen diferentes requisitos de sincronización y diferentes modos de fallo. Un diseño de un solo hilo las acoplaría: una publicación lenta de MQTT retrasaría el muestreo del sensor, y viceversa.

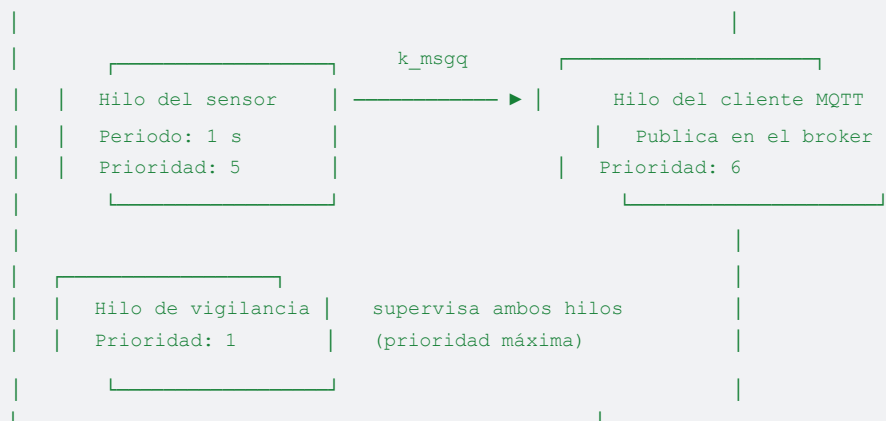
ZephyrOS proporciona un modelo multihilo preemptivo. La arquitectura de los hilos del firmware es una decisión de diseño con consecuencias para la fiabilidad, el uso de la memoria y el aislamiento de fallos.

Decisión

El firmware utiliza dos subprocesos de aplicación independientes de ZephyrOS: un subproceso de sensor y un subproceso de cliente MQTT. Se comunican a través de una cola de mensajes de ZephyrOS.

Ninguno

| Aplicación ZephyrOS |



Hilo de sensores

- **Responsabilidad:** Muestrear `el node_id`, la temperatura simulada y el estado cada 1 segundo. Poner la muestra en la cola de mensajes.
- **Prioridad:** 5 (prioridad de aplicación normal).
- **Tamaño de la pila:** 512 bytes (configurable mediante `Kconfig`).
- **Modo de fallo:** Si la cola está llena (el hilo MQTT no está consumiendo), se descarta la muestra más antigua. Esto se registra.

Hilo del cliente MQTT

- **Responsabilidad:** Descolocar muestras de la cola de mensajes y publicarlas en el broker con QoS 1. Gestionar la conexión con el broker (conectar, reconectar con retroceso, manejo de `PUBACK`).
- **Prioridad:** 6 (ligeramente inferior a la del hilo del sensor; el muestreo tiene prioridad sobre la publicación).
- **Tamaño de la pila:** 2048 bytes (la pila MQTT + TLS requiere más memoria).
- **Modo de fallo:** si no se puede contactar con el broker, el hilo vuelve a intentarlo con retroceso exponencial. El búfer fuera de línea se mantiene en la cola de mensajes.

Hilo del watchdog

- **Responsabilidad:** Verificar que tanto el hilo del sensor como el hilo MQTT se hayan registrado dentro de la ventana del watchdog (5 s). Si un hilo no se registra, el watchdog lo reinicia.
- **Prioridad:** 1 (la más alta; el watchdog debe tener prioridad sobre todos los demás hilos para hacer cumplir el tiempo de espera de registro).
- **Implementación:** `k_timer` se activa cada 5 segundos. Cada hilo de la aplicación llama a `watchdog_checkin(thread_id)` al inicio de su bucle principal.

Comunicación entre subprocesos

Se utiliza un `k_msgq` (cola de mensajes) de ZephyrOS para pasar muestras del hilo del sensor al hilo MQTT. Esto desacopla los dos hilos: el hilo del sensor nunca se bloquea por una publicación MQTT lenta.

```
C/C++  
  
// Capacidad de la cola: 60 muestras (60 segundos de telemetría a 1 s/muestra)  
K_MSGQ_DEFINE(telemetry_queue, sizeof(struct eop_telemetry), 60, 4);
```

Alternativas consideradas

Alternativa	Motivo del rechazo
Un solo hilo (secuencial)	Vincula el periodo de muestreo a la latencia de publicación de MQTT. Un tiempo de espera de red retrasa la siguiente muestra. No es adecuado para un sistema de telemetría en tiempo real.
El hilo del sensor llama directamente a la publicación MQTT	Mismo acoplamiento que el hilo único si la publicación se bloquea. También elimina el búfer fuera de línea: las muestras se pierden si no se puede contactar con el broker.
MQTT asíncrono de Zephyr (sin bloqueo)	Disponible, pero significativamente más complejo de implementar correctamente. El modelo de dos subprocesos con una cola es más sencillo y logra el mismo desacoplamiento.

Consecuencias

- Presupuesto total de RAM para los hilos de la aplicación: ~3 KB de pila. Dentro de la SRAM disponible del ESP32 tras la asignación del kernel de ZephyrOS y la pila WiFi.
 - El tamaño de la cola de mensajes (60 entradas) determina la capacidad de almacenamiento en búfer sin conexión. Una entrada por segundo → 60 segundos de tolerancia sin conexión. Esto cumple con el requisito de US-402.
 - La prioridad del hilo del watchdog (1) es la más alta de la aplicación. Esto significa que el watchdog siempre puede interrumpir cualquier hilo de la aplicación que se comporte de forma incorrecta. Los hilos del kernel de ZephyrOS tienen prioridad 0 y no pueden ser interrumpidos por el código de la aplicación.
 - Los ingenieros deben demostrar el comportamiento del watchdog: inyectar un fallo en el hilo → el watchdog detecta la falta de registro → el hilo se reinicia → la telemetría se reanuda. Este es un criterio de aceptación obligatorio (US-403, AC-6).
 - Las prioridades de los hilos y los tamaños de la pila se definen en `prj.conf` y `Kconfig`. Son configurables sin cambiar el código fuente.
-

— Fin del índice de ADR — Plataforma Exodus Ops (EOP)
Exodus Systems Inc. · División de Ingeniería · Solo para uso interno Todos los
ADR se almacenan en [/docs/adr/](#) en el repositorio del proyecto.
A